

Fondamenti di Programmazione

Alessio Avarappattu

Appunti basati sul corso tenuto dal Prof. Marco Cococcioni (@Unipi).

alessioavara.it

Diritto d'autore e Licenza

Documento redatto da Alessio Avarappattu.
sito web: alessioavara.it

Distribuito con Licenza
CC BY-NC-ND 4.0.

È consentita la condivisione e la riproduzione di questo materiale, a condizione di citare l'autore e di non utilizzarne il contenuto per scopi commerciali.

Indice

1	Rappresentazione dei Dati	12
1.1	Conversione Decimale $\rightarrow$ Binario (Algoritmo Div&Mod)	12
1.1.1	Implementazione C++ (Bit più significativo per primo)	12
1.2	Numeri Interi: Complemento a 2 (CA2)	12
1.2.1	Esercizio: Range e Somma (30 e -3)	13
1.2.2	Esercizio: Analisi Overflow ($A + B$ vs $A + B'$)	13
1.3	Numeri Reali: IEEE 754 e Half Precision	13
1.3.1	Esercizio: Rappresentazione di -1.5 (Half Precision)	13
1.3.2	Esercizio: Decodifica Half Precision	14
1.3.3	Limiti e Casi Notevoli	14
1.4	Notazione in Eccesso (Bias)	14
1.4.1	Esercizio: Rappresentazione di un numero periodico (-0.2)	14
1.5	Sottrazione in Binario (CA2)	15
1.5.1	Regole di Overflow per la Sottrazione	15
1.5.2	Esempio Svolto: Overflow	15
1.5.3	Esercizio: Rappresentazione 0.3 (Periodico Positivo)	16
1.6	Somma di Numeri Naturali (Unsigned)	16
1.6.1	Esercizio Svolto: Overflow Unsigned	16
1.7	Riepilogo Formati IEEE 754	17
1.8	Analisi Floating Point: Casi Notevoli	17
1.8.1	Esercizio di Decodifica	17
1.8.2	Lo Zero e il Minimo Assoluto	17
1.9	Gestione Stringhe C: strcpy vs strncpy	17
1.10	Aritmetica dei Puntatori	18
1.10.1	Operazioni Concesse e Vietate	18
1.10.2	Esempio Pratico (Double)	18
1.11	Errori Comuni e Riferimenti ai Puntatori	18
1.11.1	Assegnazione Diretta (Errore)	18
1.11.2	Riferimento a un Puntatore	19
1.12	Enumerazioni (enum)	19
1.13	Array Decay e sizeof	19
1.14	Aritmetica dei Char e Promozione Intera	20
1.15	Il Distruttore	20
1.15.1	Caratteristiche	20
1.15.2	Quando viene eseguito?	20
1.16	Function Overloading	21
1.16.1	Regole della Firma	21
1.16.2	Esempio Completo	21
1.16.3	Ambiguità (Ambiguity)	22
1.17	Il Ciclo di Vita degli Oggetti (Q11)	22
1.18	Errori di Memoria	22
1.19	Calcolo Inverso Notazione Eccesso (Q19)	22
1.20	I 4 Tipi Fondamentali di costruttori in C++	23
1.20.1	1. Costruttore di Default	23
1.20.2	2. Costruttore Parametrico	23
1.20.3	3. Costruttore di Copia (Copy Constructor)	23

1.20.4	4. Costruttore di Conversione	23
1.21	Effetti Collaterali (Side Effects)	24
1.22	Analisi dei Side Effects Comuni	24
2	Direttive al Preprocessore	25
2.1	Le Direttive Principali	25
2.2	Esempi di Utilizzo	26
2.2.1	1. Inclusione di File (#include)	26
2.2.2	2. Macro (#define)	26
2.2.3	3. Header Guards (Guardie di Inclusione)	26
2.3	Errore Comune: Assegnazione vs Confronto	26
2.4	Membri Statici (Static Members)	27
2.5	Riguardo ai Membri Statici	28
2.6	Condivisione di Costanti tra File (Linkage)	28
2.6.1	1. Linkage Interno (Default per const)	29
2.6.2	2. Linkage Esterno (extern)	29
2.6.3	3. C++17: Inline Variables	29
2.7	Riferimenti a Puntatori e Nullptr	29
2.7.1	1. Riferimento a un Puntatore (T*&)	29
2.7.2	2. Dereferenziazione di nullptr	30
2.8	Il concetto di Dereferenziazione (Indirection)	30
2.9	L-value vs R-value: Analisi di Espressioni	31
2.10	Riferimento Costante (const T&)	32
2.10.1	Principale Utilizzo: Passaggio di Parametri	32
2.10.2	Regola Speciale: Binding a R-values	32
2.10.3	Tabella Riassuntiva: Come passare i parametri?	32
3	Operatori: Bitwise vs Logici	32
3.1	1. Operatori Bitwise (Bit a Bit)	33
3.2	2. Operatori Logici (Booleani)	33
3.3	Esercizio Risolto: Bitwise vs Logico	33
3.4	Differenza tra sizeof e strlen	34
3.4.1	Esempio 1: Array Statico (Il caso classico)	34
3.4.2	Esempio 2: Array con dimensione fissa (Buffer)	34
3.4.3	Esempio 3: La Trappola del Puntatore (Importante!)	34
3.5	Input su C-String: Rischi e Soluzioni	35
3.5.1	1. Il Problema del Buffer Overflow	35
3.5.2	2. Il Problema degli Spazi	35
3.5.3	3. Le Soluzioni Sicure	35
3.6	Errore: Restituire puntatori a variabili locali	36
3.6.1	Soluzione 1: Usare static (Funzionante ma con limiti)	36
3.6.2	Soluzione 2: Allocazione Dinamica (Heap)	36
3.6.3	Soluzione 3: Passaggio per Valore (La migliore)	36
3.7	Ordine di Valutazione e Short-Circuit	36
3.7.1	1. Il Problema della Divisione per Zero	36
3.7.2	2. L'Errore Comune (Ordine Sbagliato)	36
3.7.3	3. La Soluzione Corretta (Sfruttare il Corto Circuito)	37
3.8	La Ricorsione	37
3.8.1	Traccia dell'esecuzione (n=3)	37

3.9	Controllo del Flusso: Break vs Continue	38
3.9.1	Nota sui Cicli Annidati (Nested Loops)	38
3.10	Bitwise Left Shift e Aritmetica	39
3.11	Divisione Intera	39
3.11.1	Soluzioni per ottenere la media corretta	39
3.11.2	Attenzione alla Precedenza degli Operatori	40
3.11.3	Perché usare il casting double()?	40
3.12	Esercizio: Valutazione di Espressioni Logiche	40
3.12.1	Nota: Stampare "true" e "false"	40
3.13	Confronto: Array Dinamico vs Array Statico	40
3.13.1	1. Puntatore allo Heap (new)	41
3.13.2	2. Array nello Stack	41
3.14	1. Puntatore a Dati Costanti (Il caso richiesto)	41
3.15	2. Puntatore Costante (Costante all'Indirizzo)	42
3.16	3. Costante Totale	42
3.16.1	Tabella Riassuntiva	42
4	Scope, Shadowing e Tempo di Vita	42
4.1	Il Fenomeno del Shadowing (Mascheramento)	42
4.2	Confronto dei Tempi di Vita (Lifetime)	43
4.3	Conversione Implicita Intero-Booleano	43
4.3.1	Analisi Caso 1: Valutazione Completa	43
4.3.2	Analisi Caso 2: Cortocircuito	43
4.4	Precedenza degli Operatori ed Errori di Runtime	44
4.4.1	Caso 1: L'errore del Modulo Zero	44
4.4.2	Caso 2: Calcolo Passo-Passo	44
4.5	Errore Comune: Il Casting "Tardivo"	44
4.5.1	Analisi dell'Errore	44
4.5.2	La Soluzione Corretta	45
4.6	I Puntatori in C++	45
4.7	Esempio di Utilizzo	45
4.8	Il Puntatore Nullo (nullptr)	45
4.9	Dichiarazione e Inizializzazione dei Puntatori	46
4.9.1	Sintassi di Base	46
4.9.2	Inizializzazione	46
4.9.3	L'Errore della Dichiarazione Multipla	46
5	Aritmetica dei Puntatori e Matrici	46
5.1	1. Il concetto di "Passo" (Stride)	46
5.2	2. Equivalenza Array-Puntatore	46
5.3	3. Linearizzazione delle Matrici (Row-Major)	47
5.3.1	Accesso tramite Puntatori (La formula esplicita)	47
6	Gestione dei File in C++	47
6.1	1. Scrittura su File (ofstream)	47
6.2	2. Lettura da File (ifstream)	48
6.3	3. Gestione degli Errori e Stati dello Stream	48
6.3.1	Il Loop di Lettura Sicuro	48

7	Gestione della Memoria Dinamica (Heap)	49
7.1	Operatori new e delete	49
7.1.1	1. Singola Variabile	49
7.1.2	2. Array Dinamici	49
7.2	Problemi Comuni	49
8	Parametri Formali vs Parametri Attuali	50
8.1	Definizioni	50
8.2	Esempio Pratico	50
8.3	Tabella di Confronto	50
9	Passaggio Parametri: Valore vs Riferimento	50
9.1	1. Passaggio per Valore (Pass-by-Value)	50
9.2	2. Passaggio per Riferimento (Pass-by-Reference)	51
9.3	Tabella Comparativa	51
10	Classi di Memorizzazione (Storage Classes)	51
10.1	1. Variabili Automatiche (Default)	51
10.2	2. La keyword <code>static</code>	52
10.2.1	A. Static Locale (dentro una funzione)	52
10.2.2	B. Static Globale (fuori dalle funzioni)	52
10.3	3. La keyword <code>extern</code>	52
10.4	4. La keyword <code>mutable</code>	52
10.5	Tabella Riassuntiva	53
10.6	Distinzione Puntatore vs Oggetto	53
10.7	Tabella Completa delle 4 Durate (C++ Standard)	53
11	L-value e R-value	54
11.1	Definizioni Fondamentali	54
11.2	1. Prefisso e PostFisso, Differenza Semantica	54
11.3	2. Esempio di Codice	54
11.4	3. Performance e Best Practice	54
12	Operatori Logici e Short-Circuit	55
12.1	1. Tabella degli Operatori	55
12.2	2. Precedenza degli Operatori	55
12.3	3. Short-Circuit Evaluation (Corto Circuito)	55
13	Matrici (Array Multidimensionali)	55
13.1	1. Definizione e Dichiarazione	55
13.2	2. Accesso agli elementi	56
13.3	3. Iterazione (Cicli Annidati)	56
13.4	4. Layout in Memoria (Row-Major)	56
14	La Grammatica del Linguaggio	56
14.1	1. Definizione	56
14.2	2. Sintassi vs Semantica	57

15 Numeri Reali (Floating Point)	57
15.1 1. Standard IEEE 754	57
15.2 2. Tipi comuni	57
15.3 3. Problemi di Approssimazione	57
16 Tipi Enumerati (enum)	57
16.1 1. Definizione	57
16.2 2. Enum Classico (C-Style)	58
16.3 3. Enum Class (C++11 - Strongly Typed)	58
17 Dimensione e Range dei Tipi Primitivi	58
17.1 1. Tabella delle dimensioni (Architettura 64-bit tipica)	58
17.2 2. Signed vs Unsigned	58
17.3 3. Ottenere i limiti via codice	59
18 Linkage Interno ed Esterno	59
18.1 1. Concetto di Unità di Traduzione	59
18.2 2. Internal Linkage (Visibilità Locale)	59
18.3 3. External Linkage (Visibilità Globale)	60
18.4 4. Errore Comune: One Definition Rule (ODR)	60
18.5 1. La regola generale (Variabili non-const)	60
18.6 2. L'eccezione del C++ (Variabili const)	60
18.7 3. Come forzare una const a essere External	60
19 L'istruzione typedef e gli Alias	61
19.1 1. Definizione	61
19.2 2. Esempi di utilizzo	61
19.2.1 A. Semplificazione di tipi lunghi	61
19.2.2 B. Chiarezza Semantica	61
19.2.3 C. Puntatori (e sicurezza)	61
19.3 3. typedef vs #define	62
19.4 4. C++ Moderno: la keyword using	62
20 Il Metalinguaggio	62
20.1 1. Definizione Concettuale	62
20.2 2. Esempi nel mondo reale	62
20.3 3. Esempio pratico: BNF	62
21 Classificazione dei Tipi di Dato	63
21.1 1. Tipi Predefiniti (Fondamentali)	63
21.2 2. Tipi Derivati (Composti)	63
21.3 3. Tipi Definiti dall'Utente (UDT)	63
21.4 4. Nota Bene: Le Stringhe	63
22 I Literals (Costanti Letterali)	63
22.1 1. Definizione	63
22.2 2. Tipi di Literal e Prefissi (Basi Numeriche)	64
22.3 3. Suffissi per forzare il tipo	64
22.4 4. Literal Stringa e Carattere	64

23 Conversione Implicita (Coercion)	64
23.1 1. Definizione	64
23.2 2. Promozione (Widening - Sicura)	65
23.3 3. Restringimento (Narrowing - Rischiosa)	65
23.4 4. Conversioni Aritmetiche Usuali	65
23.5 5. Il caso del bool	65
24 Strumenti per Programmare: Il Toolchain	65
24.1 1. Componenti Fondamentali	65
24.2 2. IDE (Integrated Development Environment)	66
24.3 3. Il Processo di Build (Compilazione)	66
25 Il Linker (Collegatore)	66
25.1 1. Definizione e Scopo	66
25.2 2. Input e Output	66
25.3 3. Funzioni Principali	66
25.4 4. Errori Tipici (Linker Errors)	67
26 Approccio Compilato vs Interpretato	67
26.1 1. Linguaggi Compilati (es. C++)	67
26.2 2. Linguaggi Interpretati (es. Python)	67
26.3 3. Tabella Riassuntiva	67
26.4 1. Errori di Compilazione (Syntax Errors)	68
26.5 2. Errori di Runtime (Execution Errors)	68
26.6 3. Errori Logici (Logic Errors / Bugs)	68
27 Programmazione Modulare	68
27.1 1. Definizione	68
27.2 2. Struttura dei File in C++	69
27.3 3. Header Guards (Guardie di inclusione)	69
27.4 4. Vantaggi	69
28 Operatori di Shift (Scorrimento)	69
28.1 1. Concetto Generale	69
28.2 2. Left Shift (◀)	69
28.3 3. Right Shift (▶)	70
29 Differenza tra #define, const e inline	70
29.1 1. Costanti: #define vs const	70
29.2 2. Funzioni: Macro vs Inline	70
29.3 3. Il pericolo dei Side Effects nelle Macro	70
30 Scope, Namespace e Visibilità	71
30.1 1. Scope (Ambito)	71
30.2 2. Namespace (Spazio dei nomi)	71
30.3 3. Operatore di Risoluzione di Scope (::)	71
30.4 4. La direttiva using	72

31 Streams e Manipolazione dell'Input/Output	72
31.1 1. Definizione di Stream	72
31.2 2. Gli Oggetti Standard (<iostream>)	72
31.3 3. Manipolazione del Formato (<iomanip>)	72
31.4 4. Persistenza dei Manipolatori (Sticky vs Non-Sticky)	73
32 Overloading degli Operatori di Input/Output	73
32.1 1. L'Obiettivo	73
32.2 2. La Regola Fondamentale	73
32.3 3. Sintassi e Pattern Standard	73
32.4 4. Punti Chiave	74
33 C-Strings: strlen, sizeof e Input	74
33.1 1. Layout in Memoria	74
33.2 2. strlen vs sizeof	74
33.3 3. Il problema del cin (Doppio Input)	75
34 Problemi di Buffer e Soluzioni	75
34.1 1. Il Buffer Condiviso	75
34.2 2. Soluzione: cin.ignore()	75
34.3 3. La Formula Magica	76
35 Overloading dell'Operatore di Assegnamento (operator=)	76
35.1 1. Perché ridefinirlo?	76
35.2 2. Lo Schema dei 4 Passaggi (The Pattern)	76
35.3 3. Regola	77
36 NULL vs nullptr	77
36.1 1. Il problema di NULL	77
36.2 2. La soluzione: nullptr	77
36.3 3. Best Practice	78
37 La Keyword friend	78
37.1 1. Definizione	78
37.2 2. Sintassi	78
37.3 3. Caratteristiche Fondamentali (Regole)	78
37.4 4. Quando usarla?	78
38 Lista di Inizializzazione (Member Initialization List)	79
38.1 1. Sintassi	79
38.2 2. Differenza con l'Assegnamento	79
38.3 3. Quando è OBBLIGATORIA?	79
38.4 4. Ordine di Inizializzazione (Trappola)	80
39 Il Puntatore this	80
39.1 1. Definizione	80
39.2 2. Principali Utilizzi	80
39.3 3. La Limitazione dei Metodi Statici	81

40	Puntatori a Oggetti e Layout di Memoria	81
40.1	1. Il Concetto di "Base Address"	81
40.2	2. Layout Sequenziale (Esempio Pratico)	81
40.3	3. Come funziona l'accesso ai membri (->)	81
40.4	4. Dimensione del Puntatore vs Oggetto	82
41	Contiguità e Data Alignment (Padding)	82
41.1	1. La Regola Generale	82
41.2	2. Il Padding (Spazio Vuoto)	82
41.3	3. Ottimizzazione (Riordinamento)	82
41.4	4. Puntatori come Membri	82
41.5	Il Padding Finale (Tail Padding)	83
42	Overloading e Tipi Primitivi	83
42.1	1. La Regola di Base	83
42.2	2. Wrapper Class	83
43	Operatore di Assegnamento e Self-Assignment	84
43.1	1. Il Problema dell'Auto-Assegnazione (Aliasing)	84
43.2	2. Il Test Standard	84
43.3	3. L'alternativa "Copy-and-Swap" (Strong Exception Safety)	84
44	Funzioni Membro Const	85
44.1	1. Significato	85
44.2	2. La Matrice di Compatibilità	85
44.3	3. Overloading su Const	86
44.4	4. Dietro le Quinte (Il Puntatore this)	86
45	Operatori di Conversione (User-Defined Conversion)	86
45.1	1. Definizione	86
45.2	2. Sintassi	86
45.3	3. Il problema delle Conversioni Implicite	87
45.4	4. La Keyword explicit (C++11)	87
46	Analisi Errore: Const Object vs Non-Const Method	87
46.1	1. Il Codice Incriminato	87
46.2	2. L'Errore	88
46.3	3. La Soluzione	88
47	Membri Statici (Static Members)	88
47.1	1. Variabili Statiche (Data Members)	88
47.2	2. Metodi Statici (Function Members)	89
48	Significati della keyword 'static'	89
48.1	1. A livello di File (Variabili/Funzioni Globali)	89
48.2	2. A livello Locale (Dentro una Funzione)	89
48.3	3. A livello di Classe	90

49 Return by Reference e Dangling References	90
49.1 1. L'Errore del Riferimento a Locale	90
49.2 2. Funzioni come L-Value	90
49.3 3. Regola di Sicurezza	90
50 Classi vs Struct in C++	90
50.1 1. Definizione	90
50.2 2. Differenze Tecniche	91
50.3 3. Esempio Comparativo	91
50.4 4. Convenzioni d'Uso	91
51 Polimorfismo in C++	91
51.1 1. Polimorfismo Statico (Early Binding)	91
51.2 2. Polimorfismo Dinamico (Late Binding)	92
51.3 3. Tabella Comparativa	92
52 Quando ritornare un Reference (T&)	92
52.1 1. Per creare un L-Value (Accesso in Scrittura)	92
52.2 2. Per il "Method Chaining" (Cascata)	93
52.3 3. Per Ottimizzazione (Read-Only)	93
52.4 4. IL DIVIETO ASSOLUTO	93
53 Algoritmo di Overload Resolution	94
53.1 1. Individuazione dei Candidati	94
53.2 2. Funzioni Viabili (Viable Functions)	94
53.3 3. Selezione della Migliore (Best Match)	94
53.4 4. Ambiguità	94
54 Strutture Dati e Algoritmi	94
54.1 3. Lista Doppia e Stampa Ricorsiva Inversa	94
54.1.1 1. La Struttura (Nodo Doppio)	94
54.1.2 2. La Funzione Ricorsiva (Il "Trucco")	95
54.1.3 3. Implementazione Completa e Test	95
54.1.4 Analisi della Complessità	96
54.2 Min e Max in Lista Ordinata	96
54.2.1 Gestione Lista Vuota	96
54.2.2 Implementazione	96
54.2.3 Ottimizzazione Possibile	97
55 Selection Sort	97
55.1 1. Principio di Funzionamento	97
55.2 2. Implementazione C++	98
55.3 3. Analisi della Complessità	98
56 Bubble Sort	98
56.1 1. Principio di Funzionamento	98
56.2 2. Implementazione C++ (Ottimizzata)	99
56.3 3. Analisi della Complessità	99
56.4 Ricerca Lineare vs Ricerca Binaria	99

56.4.1	1. Ricerca Lineare (Sequenziale)	99
56.4.2	2. Ricerca Binaria (Dicotomica)	100
56.4.3	3. Confronto Prestazionale	100
56.5	Capacità Coda Circolare (Il Caso v[10])	101
56.5.1	Domanda 1: Quanti elementi si possono memorizzare?	101
56.5.2	Domanda 2: Perché?	101
56.5.3	Domanda 3: Cosa fare per memorizzarne 10?	101
57	Definizione di Algoritmo	102
57.1	5 Proprietà (F.A.R.G.D.)	102
58	Gestione File (<fstream>)	102
58.1	Scrittura Formattata	102
58.2	Lettura Riga per Riga (Pattern Standard)	102
58.3	Modalità di Apertura (File Modes)	103
59	Ereditarietà (Inheritance)	103
59.1	1. Specificatori di Accesso all'Ereditarietà	103
59.2	2. Costruttori e Distruttori	103
59.3	3. Polimorfismo e Override	104
59.4	4. Classi Astratte e Metodi Puri	104
60	Template e Standard Template Library (STL)	104
60.1	1. I Template (Programmazione Generica)	104
60.2	2. Contenitori STL Principali	105
60.2.1	A. Vector vs List (Confronto Critico)	105
60.2.2	B. Container Adapters (Stack e Queue)	105
60.2.3	Esempio di utilizzo Vector	105
61	Sistemi di Tipizzazione	106
61.1	1. Classificazione	106
61.2	2. Il Caso C++	106
61.2.1	Type Inference (C++11 'auto')	106
61.2.2	RTTI (Run-Time Type Information)	106
62	Incapsulamento e Funzioni Virtuali	106
62.1	1. Incapsulamento (Information Hiding)	106
62.2	2. Funzioni Virtuali (Meccanismo)	107
62.2.1	Come funziona: La V-Table	107
62.2.2	Keywords Correlate (C++11)	107

1 Rappresentazione dei Dati

1.1 Conversione Decimale → Binario (Algoritmo Div&Mod)

L'algoritmo si basa sulla divisione ripetuta per 2.

- **MOD (%)**: Estrae il bit meno significativo (Resto).
- **DIV (/)**: "Shifta" il numero a destra (Divisione intera).
- **Nota**: I bit vengono generati dal LSB (Least Significant Bit) al MSB. Vanno stampati al contrario!

1.1.1 Implementazione C++ (Bit più significativo per primo)

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int numero;
6     int binario[32];
7     int i = 0;
8
9     cout << "Inserisci numero: ";
10    cin >> numero;
11
12    if (numero == 0) { cout << "0"; return 0; }
13
14    // Fase di calcolo (Ordine inverso)
15    while (numero > 0) {
16        binario[i] = numero % 2;
17        numero = numero / 2;
18        i++;
19    }
20
21    // Fase di stampa (Dall'ultimo al primo)
22    for (int j = i - 1; j >= 0; j--) {
23        cout << binario[j];
24    }
25    return 0;
26 }
```

1.2 Numeri Interi: Complemento a 2 (CA2)

È il metodo standard per i numeri con segno.

- **Range su N bit**: $[-2^{N-1}, +2^{N-1} - 1]$
- **Conversione Negativa**:
 1. Scrivi il numero positivo in binario.
 2. Inverti tutti i bit (NOT).
 3. Aggiungi 1.
- **Regola Overflow**: Si verifica SOLO sommando due numeri con lo *stesso segno* e ottenendo un risultato di segno *opposto*.

1.2.1 Esercizio: Range e Somma (30 e -3)

Problema: Rappresentare 30 e -3 su 5 bit e su 6 bit, poi sommarli.

Svolgimento:

1. **Verifica 5 bit:** Range $[-16, +15]$. Il numero 30 **non entra** (Overflow).
2. **Verifica 6 bit:** Range $[-32, +31]$. OK.
3. **Conversioni (6 bit):**

- $+30 \rightarrow 011110$
- $-3 \rightarrow (\text{inv. di } 000011 + 1) \rightarrow 111101$

4. **Somma:**

$$\begin{array}{r} 011110 \quad (+30) \\ + 111101 \quad (-3) \\ \hline 011011 \quad (+27) \end{array}$$

Risultato corretto, nessun overflow (carry-out ignorato).

1.2.2 Esercizio: Analisi Overflow ($A + B$ vs $A + B'$)

Dati $A = 10001110$ (-114) e $B = 10011100$ (-100) su 8 bit.

- **$A + B$:** Somma di due negativi $\rightarrow$ Risultato 00101010 (Positivo).
- **Commento: OVERFLOW!** Impossibile che Neg + Neg dia Pos.
- **$A + B'$ (con $B' = 11111100$):** Somma di -114 e -4.
- Risultato 10001010 (-118). **Corretto**, rientra nel range.

1.3 Numeri Reali: IEEE 754 e Half Precision

Struttura: Segno (S), Esponente (E), Mantissa (M).

$$Valore = (-1)^S \times (1.M) \times 2^{(E-Bias)}$$

Standard	Bit Totali	Bias Esponente
Half Precision	16	15 ($2^{5-1} - 1$)
Single Precision	32	127 ($2^{8-1} - 1$)

Tabella 1: Differenze tra standard

1.3.1 Esercizio: Rappresentazione di -1.5 (Half Precision)

1. **Segno:** Negativo $\rightarrow S = 1$.
2. **Binario:** $1.5_{10} = 1.1_2$.
3. **Normalizzazione:** 1.1×2^0 (Esponente reale = 0).
4. **Esponente Biased:** $E = 0 + 15 = 15 \rightarrow 01111_2$.
5. **Mantissa:** Prendo la parte frazionaria (.1) e riempio a 10 bit $\rightarrow 1000000000$.
6. **Risultato:** 1 01111 1000000000 (Hex: BC00).

1.3.2 Esercizio: Decodifica Half Precision

Dato $R = 0|00010|1110000000$.

- $S = 0$ (Positivo).
- $E = 2$. $E_{reale} = 2 - 15 = -13$.
- $M = 1.111_2$ (con hidden bit) $= 1 + 0.5 + 0.25 + 0.125 = 1.875$.
- **Valore:** 1.875×2^{-13} .

1.3.3 Limiti e Casi Notevoli

- **Zero:** Esponente 0, Mantissa 0.
- **Minimo Normalizzato (Half):** 2^{-14} (Exp=1).
- **Minimo Denormalizzato (Half):** 2^{-24} (Exp=0, Mant=0..01).

1.4 Notazione in Eccesso (Bias)

Utilizzata per gli esponenti o interi specifici.

- **Formula:** $Valore_{mem} = Valore_{reale} + Bias$.
- **Esercizio:** Rappresentare -5 su 4 bit (Eccesso $2^{4-1} = 8$).
- Calcolo: $-5 + 8 = 3$.
- Binario di 3: 0011.

1.4.1 Esercizio: Rappresentazione di un numero periodico (-0.2)

Problema: Convertire -0.2 in Half Precision. Attenzione: 0.2 è periodico in binario!

1. **Segno:** Negativo $\rightarrow S = 1$.

2. **Conversione:**

- $0.2 \times 2 = 0.4 \rightarrow 0$
- $0.4 \times 2 = 0.8 \rightarrow 0$
- $0.8 \times 2 = 1.6 \rightarrow 1$
- $0.6 \times 2 = 1.2 \rightarrow 1$ (Il ciclo 0011 si ripete).

Binario grezzo: 0.00110011...

3. **Normalizzazione:** Sposto la virgola di 3 posti:

$$1.10011... \times 2^{-3}$$

4. **Esponente Biased:** $E = -3 + 15 = 12 \rightarrow 01100_2$.

5. **Mantissa (Taglio a 10 bit):** Prendo i primi 10 bit dopo la virgola: 1001100110.

6. Risultato Finale:

1 01100 1001100110 (Hex: B266)

Nota: Poiché abbiamo troncato la mantissa, il numero salvato non è esattamente -0.2 ma un'approssimazione.

1.5 Sottrazione in Binario (CA2)

La regola fondamentale è trasformare la sottrazione in somma:

$$A - B \longrightarrow A + (-B)$$

1. Calcola il CA2 del secondo operando (B).
2. Esegui la somma binaria standard.
3. Ignora il riporto finale (Carry-out).

1.5.1 Regole di Overflow per la Sottrazione

L'Overflow si verifica quando il risultato eccede il range rappresentabile. Nella sottrazione $A - B$:

Segni Operandi	Operazione equivalente	Overflow SE...
Positivo - Negativo	$(+) + (+)$	Risultato è Negativo
Negativo - Positivo	$(-) + (-)$	Risultato è Positivo
Segni Uguali	$(+) - (+)$ o $(-) - (-)$	MAI

1.5.2 Esempio Svolto: Overflow

Calcolare $-5 - 4$ su 4 bit (Range $[-8, +7]$).

- $A = -5 \rightarrow 1011$
- $B = 4 \rightarrow 0100$
- Operazione: Sommare il CA2 di B .
- CA2 di 4: $\text{inv}(0100) + 1 \rightarrow 1011 + 1 = 1100$ (-4).
- Somma: $1011(-5) + 1100(-4)$.

$$\begin{array}{r} 1011 \\ + 1100 \\ \hline 1\ 0111 \quad (\text{Carry ignorato}) \end{array}$$

Risultato: 0111 che vale +7. **Analisi:** Doveva fare -9. Ho fatto (Neg - Pos) e ho ottenuto Positivo. **OVERFLOW.**

1.5.3 Esercizio: Rappresentazione 0.3 (Periodico Positivo)

Problema: Convertire 0.3 in Half Precision.

1. **Segno:** Positivo $\rightarrow S = 0$.

2. **Conversione:**

- $0.3 \times 2 = 0.6 \rightarrow 0$
- $0.6 \times 2 = 1.2 \rightarrow 1$ (resto 0.2)
- $0.2 \times 2 = 0.4 \rightarrow 0$ (Inizia il ciclo 0011)

Binario: 0.0100110011...

3. **Normalizzazione:** Sposto la virgola di 2 posti a destra:

$$1.00110011... \times 2^{-2}$$

4. **Esponente:** $E = -2 + 15 = 13 \rightarrow 01101_2$.

5. **Mantissa (10 bit):** 0011001100.

6. **Risultato:**

$$0 \ 01101 \ 0011001100 \quad (\text{Hex: } 34CC)$$

1.6 Somma di Numeri Naturali (Unsigned)

Quando si lavora con numeri **Naturali** (senza segno), tutti i bit sono usati per il valore (magnitudine).

- **Range su N bit:** $[0, 2^N - 1]$.
- **Regola Overflow:** Si ha errore se la somma genera un **Riporto in Uscita (Carry-Out = 1)** dal bit più significativo (MSB).

1.6.1 Esercizio Svolto: Overflow Unsigned

Dati $A = 10111000$ e $B = 10001011$ su 8 bit.

1. **Conversione Decimale:**

- $A = 128 + 32 + 16 + 8 = 184$.
- $B = 128 + 8 + 2 + 1 = 139$.
- Somma attesa: $184 + 139 = 323$.

2. **Calcolo Binario:**

$$\begin{array}{r} 10111000 \\ + 10001011 \\ \hline 1\ 01000011 \end{array}$$

3. **Analisi:** Il risultato su 8 bit è 01000011 (67_{10}).

4. **Conclusione:** Poiché $323 > 255$ (max su 8 bit) e c'è un **Carry Out di 1**, il risultato è **ERRATO** (Overflow).

1.7 Riepilogo Formati IEEE 754

Formato	Bit Totali	Segno	Esponente	Mantissa	Bias
Half	16	1	5	10	15
Single	32	1	8	23	127
Double	64	1	11	52	1023

Tabella 2: Suddivisione dei bit nei vari standard

Nota sul Bit Nascosto: La mantissa memorizzata è la parte frazionaria (F). Il valore reale è $1.F$. Quindi la precisione effettiva è sempre **Bit Mantissa** + 1.

1.8 Analisi Floating Point: Casi Notevoli

1.8.1 Esercizio di Decodifica

Dato un formato con 5 bit di Esponente (Bias 15) e 8 bit di Mantissa:

$$S = 1, \quad E = 11101, \quad F = 11100000$$

1. **Segno:** $1 \rightarrow$ Negativo.
2. **Esponente:** $29 - 15 = 14$.
3. **Mantissa:** 1.111_2 (con hidden bit).
4. **Calcolo:** $-1.111_2 \times 2^{14} = -11110000000000_2$.
5. **Decimale:** $-(16384 + 8192 + 4096 + 2048) = -30720$.

1.8.2 Lo Zero e il Minimo Assoluto

- **Lo Zero:** È un caso speciale definito da $E = 0$ e $F = 0$. Non si calcola con la formula standard.
- **Numero più vicino a 0 (Denormalizzato):** Quando $E = 0$ ma $F \neq 0$, il numero diventa **Denormalizzato**:

$$Valore = (-1)^S \times (0.F) \times 2^{1-Bias}$$

Il più piccolo numero rappresentabile (in valore assoluto) ha $F = 00...01$.

$$\text{Min} = 2^{-\text{bit_mantissa}} \times 2^{-14}$$

1.9 Gestione Stringhe C: strcpy vs strncpy

Le stringhe in stile C sono array di caratteri terminati da `'\0'`.

<code>strcpy(dest, src)</code>	<code>strncpy(dest, src, n)</code>
Copia caratteri finché non trova <code>'\0'</code> .	Copia al massimo n caratteri.
Rischio: Buffer Overflow se src è più lunga di dest .	Previene il Buffer Overflow limitando la copia.
Copia sempre il terminatore <code>'\0'</code> .	Attenzione: Se src $\geq$ n , NON aggiunge <code>'\0'</code> alla fine.

```

1 char buffer[10];
2 // Copio max 9 char per lasciare spazio al terminatore
3 strncpy(buffer, "StringaMoltoLunga", 9);
4 buffer[9] = '\0'; // Assicuro la terminazione manuale!

```

Listing 1: Uso corretto di strncpy

1.10 Aritmetica dei Puntatori

I puntatori non sono semplici interi; le operazioni matematiche su di essi tengono conto della dimensione del tipo di dato puntato (*Scaling*).

1.10.1 Operazioni Concesse e Vietate

Dati due puntatori p , q e un intero N :

- $p + N$: **Valido**. L'indirizzo aumenta di $N \times \text{sizeof}(\text{tipo})$.
- $p - q$: **Valido**. Restituisce il numero di elementi (celle) tra i due puntatori.
- $p + q$: **ERRORE DI COMPILAZIONE**. Sommare due indirizzi non ha senso logico.

1.10.2 Esempio Pratico (Double)

Ipotizziamo che p punti all'indirizzo 1000 e che $\text{sizeof}(\text{double}) = 8$.

```

1 double* p = (double*)1000; // Esempio teorico
2
3 cout << p + 1;
4 // Stampa 1008 (1000 + 1*8)
5
6 cout << p + 3;
7 // Stampa 1024 (1000 + 3*8)

```

1.11 Errori Comuni e Riferimenti ai Puntatori

1.11.1 Assegnazione Diretta (Errore)

Il codice `int *p = 7;` genera un **Errore di Compilazione**.

- Un puntatore attende un indirizzo di memoria, non un valore intero.
- **Soluzione:** Creare una variabile e puntare al suo indirizzo.
- `int a = 7; int *p = &a;`

1.11.2 Riferimento a un Puntatore

È possibile creare un riferimento (alias) a un puntatore. Questo permette di modificare l'indirizzo contenuto nel puntatore originale passando il riferimento a una funzione.

Sintassi: Tipo* & nome_ref = nome_puntatore;

```
1 int x = 10;
2 int y = 20;
3 int* p = &x;      // p punta a x
4
5 int*& ref = p;
6
7 ref = &y;          // ORA ANCHE p PUNTA A y!
8 // Se avessi usato un puntatore normale, p non sarebbe cambiato.
```

1.12 Enumerazioni (enum)

Le enum definiscono costanti intere.

- Il primo elemento vale 0 (se non specificato).
- Gli elementi successivi valgono *Precedente + 1*.
- Se si assegna un valore esplicito, la numerazione riparte da quel valore.

```
1 enum colore {
2     ROSSO,      // 0 (Default)
3     GIALLO=5,   // 5 (Esplicito)
4     VERDE,      // 6 (5 + 1)
5     BLU         // 7 (6 + 1)
6 };
7 // Attenzione: "enum class" NON viene convertita implicitamente in int
```

1.13 Array Decay e sizeof

Un array dichiarato staticamente (`int v[10]`) mantiene la sua dimensione solo nello scope in cui è dichiarato.

- **Dichiarazione locale:** `sizeof(v)` restituisce la dimensione totale in byte (es. $10 \times 4 = 40$).
- **Passaggio a funzione:** L'array decade in un **puntatore**. La dimensione scritta tra quadre viene ignorata. `sizeof(v)` restituisce la dimensione del puntatore (4 o 8 byte).

```
1 void f(int a[]) {
2     // Qui 'a' è solo un puntatore int*
3     cout << sizeof(a); // Stampa 8 (su 64-bit)
4 }
5
6 int main() {
7     int v[10];
8     cout << sizeof(v); // Stampa 40
9     f(v);}

```

Listing 2: Il trucco del sizeof

Regola: Se vuoi passare un array a una funzione e conoscerne la dimensione, devi passare la dimensione come **secondo parametro** ('void f(int* v, int size)').

1.14 Aritmetica dei Char e Promozione Intera

I tipi `char` sono interi a 8 bit. Nelle operazioni aritmetiche, vengono promossi a `int`.

```
1 char c1 = 'a'; // ASCII 97
2 char c2 = 'c'; // ASCII 99
3
4 // 1. SOMMA MATEMATICA
5 cout << c1 + c2;
6 // Output: 196 (97 + 99). Stampa la somma degli ASCII.
7
8 // 2. STAMPARE IL CARATTERE RISULTANTE (CAST)
9 cout << (char)(c1 + c2);
10 // Output: Il simbolo corrispondente al codice 196.
11
12 // 3. CONCATENAZIONE VISIVA
13 cout << c1 << c2;
14 // Output: ac (Stampa prima uno, poi l'altro).
```

Ricorda:

- `'a' + 'b'` = Somma di Interi (es. 195).
- `"a" + "b"` = Errore (non puoi sommare due array di `char` puri).
- `string("a") + string("b")` = `"ab"` (Concatenazione vera).

1.15 Il Distruttore

Il distruttore è un metodo speciale invocato quando un oggetto viene distrutto. Il suo scopo principale è prevenire i **Memory Leak** rilasciando le risorse allocate dinamicamente.

1.15.1 Caratteristiche

- **Sintassi:** `~NomeClasse()`.
- Non ha parametri, non ha tipo di ritorno, non può essere sovraccaricato (ne esiste uno solo per classe).

1.15.2 Quando viene eseguito?

1. **Oggetti Statici/Automatici (Stack):** Viene chiamato automaticamente quando la variabile esce dallo scope (chiusura graffa `}`).
2. **Oggetti Dinamici (Heap):** Viene chiamato **solo** quando si esegue l'operatore `delete` sul puntatore.

```
1 class Buffer {
2     int* ptr;
3 public:
4     Buffer() {
5         ptr = new int[100]; // Acquisisco risorsa
```

```

6     }
7
8     // Il distruttore evita il Memory Leak
9     ~Buffer() {
10         delete[] ptr;        // Rilascio risorsa
11     }
12 };
13
14 void test() {
15     Buffer b; // Costruttore chiamato
16 } // Qui 'b' muore -> Distruttore chiamato automaticamente

```

Listing 3: Esempio di gestione memoria

1.16 Function Overloading

L'overloading consente di definire più funzioni con lo stesso identificatore, purché la loro **firma** sia diversa.

1.16.1 Regole della Firma

Il compilatore distingue le funzioni in base a:

- Numero dei parametri.
- Tipo dei parametri.

Nota Importante: Il tipo di ritorno **NON** distingue le funzioni.

```

1 int f(int a);
2 double f(int a); // ERRORE: differisce solo per il ritorno!

```

1.16.2 Esempio Completo

```

1 // 1. Somma di interi
2 int somma(int a, int b) {
3     return a + b;
4 }
5
6 // 2. Somma di double
7 double somma(double a, double b) {
8     return a + b;
9 }
10
11 // 3. Somma di tre interi
12 int somma(int a, int b, int c) {
13     return a + b + c;
14 }
15
16 int main() {
17     somma(10, 20);        // Chiama la 1
18     somma(1.5, 2.5);      // Chiama la 2
19     somma(1, 2, 3);       // Chiama la 3
20 }

```

Listing 4: Overloading della funzione somma

1.16.3 Ambiguità (Ambiguity)

L'overloading può fallire se il compilatore non sa quale scegliere (es. conversioni implicite).
Esempio: Se hai `f(int)` e `f(double)`, chiamare `f(3.5f)` (float) potrebbe essere ambiguo se non esiste una corrispondenza esatta.

1.17 Il Ciclo di Vita degli Oggetti (Q11)

La chiamata al distruttore dipende da dove risiede l'oggetto:

- **Stack (Variabili locali):** Il distruttore è **automatico**. Viene chiamato quando l'esecuzione esce dallo scope (chiusura `}`).
- **Heap (Puntatori con `new`):** Il distruttore è **manuale**. Viene chiamato **esclusivamente** se si esegue l'istruzione `delete`. Se ci si dimentica, il distruttore non parte e le risorse non vengono rilasciate correttamente.

1.18 Errori di Memoria

È fondamentale distinguere i diversi tipi di problemi:

Errore	Descrizione e Causa
Memory Leak	Memoria allocata nello Heap che non viene mai liberata con <code>delete</code> . Il programma consuma RAM inutilmente (es. <code>new</code> senza <code>delete</code>).
Buffer Overflow	Scrittura oltre i limiti di un array (es. indice 10 su array size 10). Corrompe i dati adiacenti.
Stack Overflow	Esaurimento dello spazio nello Stack. Causa tipica: Ricorsione infinita o allocazione di array locali enormi.
Segmentation Fault	Accesso illegale alla memoria. Cause: dereferenziazione di <code>nullptr</code> , puntatori non inizializzati, o accesso a memoria liberata (<i>dangling pointer</i>).

1.19 Calcolo Inverso Notazione Eccesso (Q19)

Per trovare il valore reale da un numero in notazione Eccesso- K (Bias K):

$$Valore_{reale} = Valore_{memorizzato} - Bias$$

Esempio Quiz:

- **Bias:** 4.
- **Dato Binario:** 100.
- **Valore Memorizzato:** $100_2 = 4_{10}$.
- **Calcolo:** $4 - 4 = 0$.

Nota: In eccesso, il valore binario che eguaglia il bias rappresenta sempre lo Zero.

1.20 I 4 Tipi Fondamentali di costruttori in C++

1.20.1 1. Costruttore di Default

Non accetta parametri. Se non viene definito alcun costruttore, il compilatore ne genera uno implicito.

```
1 class Classe {
2 public:
3     Classe() {
4         // Inizializzazione di default
5     }
6 };
7 // Uso: Classe obj;
```

1.20.2 2. Costruttore Parametrico

Accetta parametri per inizializzare i membri. Supporta l'overloading.

```
1 class Classe {
2     int membro;
3 public:
4     Classe(int a) : membro(a) {
5         // Corpo del costruttore
6     }
7 };
8 // Uso: Classe obj(10);
```

1.20.3 3. Costruttore di Copia (Copy Constructor)

Crea un nuovo oggetto copiando i dati da un altro oggetto dello stesso tipo. Fondamentale per la *Deep Copy* in presenza di puntatori.

- **Firma:** `Classe(const Classe& other)`
- **Uso:** `Classe obj2 = obj1;` oppure passando oggetti per valore a funzioni.

1.20.4 4. Costruttore di Conversione

Qualsiasi costruttore che può essere chiamato con un **singolo argomento** agisce implicitamente come convertitore dal tipo dell'argomento al tipo della Classe.

```
1 class Frazione {
2     int n;
3 public:
4     // Costruttore di conversione (int -> Frazione)
5     Frazione(int num) : n(num) {}
6 };
7
8 // Conversione Implicita:
9 Frazione f = 5; // Chiama Frazione(5)
```

Keyword explicit: Per evitare conversioni accidentali, si antepone `explicit` al costruttore. In tal caso, `Frazione f = 5;` darebbe errore (servirebbe `Frazione f(5);`).

1.21 Effetti Collaterali (Side Effects)

In programmazione, una funzione ha un **effetto collaterale** se, oltre a restituire un valore (o invece di restituirlo), modifica lo stato del sistema o interagisce con l'esterno.

Le 3 cause principali:

1. **Modifica di variabili globali o statiche:** La funzione altera dati che persistono oltre la sua esecuzione.
2. **Input/Output (I/O):** Operazioni come stampare a video (`std::cout`), leggere da tastiera, o scrivere su file/database.
3. **Modifica dei parametri (tramite puntatori/riferimenti):** Se un argomento è passato per riferimento non costante e viene modificato, questo è un side effect visibile al chiamante.

```
1 int contatore = 0; // Variabile globale
2
3 // Esempi di Side Effects
4 void funzioneImpura(int* val) {
5     contatore++;           // 1. Modifica stato globale
6     *val = 0;              // 2. Modifica parametro (tramite puntatore)
7     std::cout << "Ok";    // 3. I/O (modifica buffer di output)
8 }
9
10 // Funzione Pura (Referential Transparency)
11 // L'output dipende SOLO dall'input. Nessun cambiamento di stato.
12 int somma(int a, int b) {
13     return a + b;
14 }
```

Nota: Le funzioni senza effetti collaterali sono dette *Funzioni Pure*. Sono più facili da testare e permettono ottimizzazioni del compilatore.

1.22 Analisi dei Side Effects Comuni

Di seguito viene analizzato perché determinate operazioni sono classificate come effetti collaterali. Il principio guida è la **modifica di uno stato esterno allo scope locale della funzione**.

1. Modifica Variabile Globale:

```
1 int g = 0;
2 void f() { g++; } // Side Effect
3
```

La variabile `g` risiede nel segmento dati (statico), non nello stack della funzione. La funzione altera la "storia" del programma.

2. Parametri per Riferimento (Output Params):

```
1 void f(int& x) { x = 10; } // Side Effect
2
```

La funzione modifica direttamente la memoria del chiamante. L'argomento non è una copia locale, ma un alias di una variabile esterna.

3. Input/Output (I/O):

```
1 void f() { std::cout << "Hi"; } // Side Effect
2
```

Lo stream di output (console, file, rete) è una risorsa di sistema condivisa. Scrivere su di esso altera lo stato del dispositivo di output.

4. Mutazione dello Stato dell'Oggetto:

```
1 class C {
2     int v;
3     void set(int x) { v = x; } // Side Effect
4 };
5
```

Il metodo modifica `this->v`. L'istanza dell'oggetto cambia stato internamente; le chiamate future ai suoi metodi potrebbero restituire risultati diversi.

5. **Manipolazione Memoria Dinamica (Heap):** Operazioni come `new` o `delete` modificano la mappa di memoria del sistema operativo (Heap) e spesso puntatori esterni passati per riferimento.
6. **Persistenza (File System):** Scrivere su disco altera fisicamente i dati memorizzati, un cambiamento che persiste anche dopo la terminazione del processo.

2 Direttive al Preprocessore

Il **Preprocessore** è un componente che elabora il codice sorgente prima della compilazione effettiva. Le sue direttive iniziano con `#` e non terminano con `;`. Il suo compito principale è la manipolazione del testo (sostituzione, inclusione, esclusione).

2.1 Le Direttive Principali

Direttiva	Descrizione
<code>#include</code>	Inserisce il contenuto di un altro file nel punto corrente. Usato per le librerie e gli header file.
<code>#define</code>	Definisce una Macro . Sostituisce ogni occorrenza di un identificatore con una stringa di testo specifica.
<code>#undef</code>	Rimuove una definizione precedente creata con <code>#define</code> .
<code>#ifdef</code> / <code>#ifndef</code>	Compilazione Condizionale: Include il codice seguente solo se una macro è (o non è) definita.
<code>#endif</code>	Termina un blocco condizionale (<code>#if...</code>).
<code>#pragma</code>	Istruzioni specifiche per il compilatore (es. <code>#pragma once</code> per evitare inclusioni multiple).

Tabella 3: Principali direttive del preprocessore

2.2 Esempi di Utilizzo

2.2.1 1. Inclusione di File (#include)

- `#include <iostream>`: Cerca nelle directory di sistema (librerie standard).
- `#include "miofile.h"`: Cerca prima nella directory corrente del progetto.

2.2.2 2. Macro (#define)

Attenzione: Il preprocessore fa una sostituzione puramente testuale, senza controlli di tipo.

```
1 #define PI 3.14159          // Sostituisce PI con 3.14159 ovunque
2 #define QUADRATO(x) ((x)*(x)) // Macro con argomenti
3
4 int area = PI * QUADRATO(5);
5 // Diventa: int area = 3.14159 * ((5)*(5));
```

2.2.3 3. Header Guards (Guardie di Inclusione)

Fondamentali per evitare che un file header (.h) venga incluso più volte nello stesso file sorgente, causando errori di ridefinizione.

```
1 // File: MiaClasse.h
2
3 // Metodo Classico (Standard C++)
4 #ifndef MIA_CLASSE_H      // Se non e' definito...
5 #define MIA_CLASSE_H      // ...definiscilo ora e includi il codice
6
7 class MiaClasse {
8     // ...
9 };
10
11 #endif // Fine del blocco if
12
13 // Metodo Moderno (Non standard ma supportato ovunque)
14 #pragma once
```

2.3 Errore Comune: Assegnazione vs Confronto

Uno degli errori logici più frequenti in C++ è l'utilizzo dell'operatore di assegnazione (=) al posto dell'operatore di uguaglianza (==) all'interno delle condizioni.

```
1 int n = 5;
2
3 // ERRORE:
4 if (n = 2) {
5     // 1. Assegna 2 a n (n diventa 2)
6     // 2. Valuta l'espressione: risultato 2
7     // 3. 2 != 0 -> Condizione TRUE
8     cout << "Sempre eseguito";
9 }
10
11 // CORRETTO:
12 if (n == 2) {
13     // Confronta n con 2. Restituisce false (5 != 2).
```

```

14     cout << "Eseguito solo se n e' 2";
15 }

```

Best Practice (Yoda Conditions): Per evitare questo errore, alcuni programmatori scrivono la costante a sinistra (stile Yoda):

```

1 if (2 = n) // Errore di compilazione! (Impossibile assegnare a un
    letterale)
2 if (2 == n) // Corretto e sicuro

```

Tuttavia, i compilatori moderni avvertono di questo errore se si attivano i warning (-Wall).

2.4 Membri Statici (Static Members)

La keyword `static` all'interno di una classe indica che il membro appartiene alla classe stessa e non alle singole istanze.

• Variabili Statiche (Shared Data):

- Condivise tra tutti gli oggetti della classe (memoria unica).
- Devono essere **definite** (inizializzate) fuori dalla classe (nel file .cpp solitamente) per allocare spazio.
- Utili per contatori globali, configurazioni condivise o costanti di classe.

• Funzioni Statiche:

- Possono essere invocate senza istanziare un oggetto: `Classe::metodo()`.
- **Non possiedono il puntatore this.**
- Possono accedere **solo** a membri statici (non possono accedere alle variabili di istanza normali).

```

1 class Utente {
2 private:
3     int id;
4     // Dichiarazione: Esiste, ma non ha ancora memoria allocata
5     static int contatoreTotale;
6
7 public:
8     Utente() {
9         contatoreTotale++; // Ogni nuovo utente incrementa il totale
10        condiviso
11        id = contatoreTotale;
12    }
13
14    // Funzione Statica: Puo' accedere solo a contatoreTotale
15    static int getTotale() {
16        // return id; // ERRORE! Non so di quale oggetto prendere l'id
17        return contatoreTotale;
18    }
19 };
20
21 // Definizione e Inizializzazione (Obbligatoria fuori dalla classe!)
22 int Utente::contatoreTotale = 0;
23
24 int main() {

```

```

24 // Posso chiamare il metodo statico anche senza oggetti
25 cout << Utente::getTotale(); // Output: 0
26
27 Utente u1;
28 Utente u2;
29
30 cout << Utente::getTotale(); // Output: 2
31 // u1.contatoreTotale e u2.contatoreTotale sono la stessa variabile
32 }

```

Listing 5: Esempio: Contatore di Istanze

2.5 Riguardo ai Membri Statici

- **1. Definizione Esterna (ODR):** Le variabili `static` all'interno della classe sono solo *dichiarazioni*. Devono essere **definite** (e inizializzate) nel file sorgente (.cpp) globale per allocare memoria. *Eccezione:* Le costanti intere (`static const int`) possono essere inizializzate dentro la classe.
- **2. Assenza di this:** I metodi `static` non vengono invocati su un oggetto specifico, quindi non possiedono il puntatore implicito `this`. Di conseguenza:
 - Non possono accedere a variabili non statiche.
 - Non possono chiamare funzioni membro non statiche.
- **3. Sintassi di Accesso:** Si accede tramite l'operatore di risoluzione dello scope (`::`), senza bisogno di istanziare la classe. `NomeClasse::MembroStatico`.

```

1 class Esempio {
2 public:
3     static int contatore; // Dichiarazione (senza memoria)
4     int id;
5
6     static void reset() {
7         contatore = 0; // OK: Accede a membro statico
8         // id = 0; // ERRORE: 'id' richiede un oggetto (no 'this')
9         // this->id = 0; // ERRORE: 'this' non esiste qui
10    }
11 };
12
13 // DEFINIZIONE (Allocazione Memoria) - Obbligatoria!
14 int Esempio::contatore = 0;
15
16 int main() {
17     Esempio::reset(); // Accesso senza oggetto
18     Esempio::contatore = 5; // Accesso diretto
19 }

```

Listing 6: Sintassi ed Errori Comuni con static

2.6 Condivisione di Costanti tra File (Linkage)

In C++, la visibilità di una variabile tra i diversi file di traduzione (file .cpp) è determinata dal suo **Linkage**.

2.6.1 1. Linkage Interno (Default per const)

Le costanti definite globalmente (`const int A = 10;`) hanno linkage interno.

- Se definite in un header file (`.h`), ogni file `.cpp` che lo include ne crea una **copia locale**.
- Non genera errori di "Multiple Definition", ma le variabili hanno indirizzi di memoria diversi.

2.6.2 2. Linkage Esterno (extern)

Per condividere la **stessa identica variabile** (stesso indirizzo di memoria) tra più file, si usa il pattern **Dichiarazione/Definizione** con `extern`.

```
1 // --- File: Costanti.h ---
2 #ifndef COSTANTI_H
3 #define COSTANTI_H
4
5 // 1. DICHIARAZIONE (Promessa: esiste altrove)
6 extern const double GRAVITA;
7
8 #endif
9
10 // --- File: Costanti.cpp ---
11 #include "Costanti.h"
12
13 // 2. DEFINIZIONE (Allocazione memoria effettiva)
14 const double GRAVITA = 9.81;
15
16
17 // --- File: Main.cpp ---
18 #include "Costanti.h"
19 #include <iostream>
20
21 int main() {
22     // Usa la variabile definita in Costanti.cpp
23     std::cout << GRAVITA;
24 }
```

Listing 7: Condivisione corretta con extern

2.6.3 3. C++17: Inline Variables

Dallo standard C++17, è possibile definire la variabile direttamente nell'header usando `inline`. Il linker si occupa di unificare le copie.

```
1 // File: Costanti.h (Solo C++17 o sup.)
2 inline const int MAX_VITE = 3; // Unica istanza condivisa
```

2.7 Riferimenti a Puntatori e Nullptr

2.7.1 1. Riferimento a un Puntatore (T*&)

È possibile creare un riferimento che agisce come alias di una variabile puntatore. Questo permette di modificare l'indirizzo memorizzato nel puntatore originale, specialmente nel passaggio di parametri alle funzioni.

Sintassi:

```
1 int x = 10;
2 int* ptr = &x;
3
4 // refPtr e' un alias di ptr
5 int*& refPtr = ptr;
6
7 refPtr = nullptr; // Modifica anche ptr! Ora ptr vale nullptr.
```

Caso d'uso comune (Allocazione in funzione): Se passiamo un puntatore per valore (`int* p`), la funzione modifica una copia locale. Per modificare il puntatore originale (es. allocare memoria), dobbiamo passarlo per riferimento al puntatore (`int*& p`) o puntatore a puntatore (`int** p`).

```
1 // Funzione che alloca memoria per il chiamante
2 void alloca(int*& p) {
3     p = new int[10]; // Modifica il puntatore del main
4 }
5
6 int main() {
7     int* mioPtr = nullptr;
8     alloca(mioPtr); // Ora mioPtr punta al nuovo array
9     delete[] mioPtr;
10 }
```

2.7.2 2. Dereferenziazione di nullptr

Tentare di accedere al valore puntato da un puntatore nullo (`*ptr` dove `ptr == nullptr`) causa un **Undefined Behavior**.

- **Effetto pratico:** Crash del programma (Segmentation Fault / Access Violation).
- **Causa:** L'indirizzo 0x0 è riservato e protetto dal sistema operativo.

```
1 int* p = nullptr;
2
3 // ERRORE GRAVE:
4 // *p = 5; // Crash immediato! Scrittura su indirizzo 0.
5 // int x = *p; // Crash immediato! Lettura da indirizzo 0.
```

2.8 Il concetto di Dereferenziazione (Indirection)

Dereferenziare un puntatore significa **accedere al valore** memorizzato all'indirizzo di memoria puntato.

In C++, si usa l'operatore unario `*` (asterisco) davanti al nome del puntatore.

- **Il puntatore (`p`):** Contiene un indirizzo di memoria (es. 0x7ffee4).
- **La dereferenziazione (`*p`):** Segue quell'indirizzo per leggere o scrivere il valore effettivo che si trova lì.

```

1 int numero = 100;
2 int* p = &numero; // p punta all'indirizzo di 'numero'
3
4 // 1. Stampare il puntatore
5 std::cout << p;    // Output: 0x7ffd5... (L'indirizzo di memoria)
6
7 // 2. Dereferenziare (Leggere)
8 std::cout << *p;   // Output: 100 (Il valore dentro quella memoria)
9
10 // 3. Dereferenziare (Scrivere)
11 *p = 200;          // Va all'indirizzo di p e cambia il valore in 200
12 std::cout << numero; // Output: 200 (La variabile originale e' cambiata
    !)
```

Listing 8: Dereferenziazione: Dall'indirizzo al valore

Nota sulla sintassi confusa: L'asterisco ha due significati diversi in base al contesto:

1. `int* p;` → **Dichiarazione:** "Sto creando un tipo puntatore".
2. `*p = 5;` → **Operatore:** "Sto dereferenziando (accedendo al valore)".

2.9 L-value vs R-value: Analisi di Espressioni

La differenza tra queste due espressioni illustra il tipo di valore restituito dagli operatori aritmetici rispetto a quelli di assegnazione.

1. `(a - b) = 7;` → **Errore di Compilazione.**
 - L'operatore `-` restituisce un **R-value** (un valore temporaneo risultante dal calcolo).
 - Un R-value non ha un indirizzo di memoria persistente a cui assegnare un nuovo valore. È come tentare di scrivere `10 = 7;`.
2. `(a -= b) = 7;` → **Codice Valid.**
 - L'operatore `-=` restituisce un **L-value** (un riferimento alla variabile `a` modificata).
 - Poiché il risultato è un riferimento a una locazione di memoria valida (`a`), è possibile assegnargli un nuovo valore subito dopo.
 - *Esecuzione:* Prima `a` viene decrementato di `b`, poi il risultato viene sovrascritto da 7.

```

1 int a = 10, b = 2;
2
3 // (a - b) restituisce 8 (int temporaneo).
4 // 8 = 7; // ERRORE!
5
6 // (a -= b) modifica a in 8 e restituisce 'int&' (riferimento ad a).
7 // a = 7; // OK!
8 (a -= b) = 7;
```

Listing 9: Return Type degli Operatori

2.10 Riferimento Costante (const T&)

Un riferimento costante è un alias di sola lettura a una variabile esistente. Sintassi: `const Tipo& nomeRef = variabile;`

2.10.1 Principale Utilizzo: Passaggio di Parametri

È lo standard *de facto* per passare oggetti complessi (classi, stringhe, vector) alle funzioni.

- **Efficienza:** Evita la *Deep Copy* dell'oggetto (passa solo l'indirizzo).
- **Sicurezza:** Il compilatore impedisce alla funzione di modificare l'oggetto originale.

```
1 #include <vector>
2 #include <string>
3
4 // MALE: Passaggio per Valore (Copia intero vector!)
5 void elabora(std::vector<int> v) { ... }
6
7 // BENE: Passaggio per Riferimento Costante (Nessuna copia, Solo lettura)
8 void elabora(const std::vector<int>& v) {
9     // v.push_back(10); // ERRORE: v e' const!
10    int x = v[0];        // OK: Lettura permessa
11 }
```

Listing 10: Efficienza con const reference

2.10.2 Regola Speciale: Binding a R-values

A differenza di un riferimento normale (`int&`), un riferimento costante **può agganciarsi a un valore temporaneo (literal)** prolungandone la vita.

```
1 // int& r = 5;           // ERRORE: 5 e' un letterale (R-value)
2 const int& r = 5;       // OK: Il compilatore crea una variabile temporanea
                          nascosta
```

2.10.3 Tabella Riassuntiva: Come passare i parametri?

Tipo di dato	Come passarlo	Perché?
Tipi primitivi (int, double, char)	Valore (int x)	Piccoli, veloci da copiare.
Oggetti/Classi (string, vector)	Const Ref (const T& x)	Evita copie pesanti.
Oggetti da modificare	Riferimento (T& x)	Serve modificare l'originale.

3 Operatori: Bitwise vs Logici

È fondamentale distinguere tra gli operatori che manipolano i singoli bit e quelli che valutano la logica booleana.

Op	Nome	Logica (sui bit)
&	AND	1 se entrambi i bit sono 1.
	OR	1 se almeno un bit è 1.
^	XOR (Exclusive OR)	1 se i bit sono diversi (0-1 o 1-0).
~	NOT (Complemento)	Inverte tutti i bit (0→1, 1→0).
«	Left Shift	Sposta i bit a sinistra (moltiplica per 2).
»	Right Shift	Sposta i bit a destra (divide per 2).

3.1 1. Operatori Bitwise (Bit a Bit)

Agiscono sulla rappresentazione binaria dei numeri interi. Non usano il corto circuito.

Esempio XOR (^): Utile per trovare differenze o scambiare valori.

```
1 // 5 (0101) XOR 3 (0011) = 6 (0110)
2 int x = 5 ^ 3;
```

3.2 2. Operatori Logici (Booleani)

Agiscono su valori di verità (true/false). In C++, 0 è false, qualsiasi altro intero è true.

Op	Nome	Comportamento
&&	Logical AND	Vero se entrambi sono veri.
	Logical OR	Vero se almeno uno è vero.
!	Logical NOT	Inverte il valore di verità (!true = false).

Regola del Corto Circuito (Short-Circuit Evaluation):

- In A && B: Se A è **falso**, B non viene nemmeno valutato (risultato sicuramente falso).
- In A || B: Se A è **vero**, B non viene nemmeno valutato (risultato sicuramente vero).

3.3 Esercizio Risolto: Bitwise vs Logico

Dati: unsigned int a=3, b=1, c=5;

- **Bitwise:** ((a|c)&b)
 - $3_{10} = 0011_2$, $5_{10} = 0101_2$.
 - $0011 | 0101 = 0111_2$ (7_{10}).
 - $0111 \& 0001 = 0001_2$ (1_{10}).
- **Logico:** ((a||c) && b)
 - $(3 \vee 5)$ è true (perché non sono zero).
 - $(\text{true} \wedge 1)$ è true.
 - Output: **1**.

3.4 Differenza tra sizeof e strlen

Sebbene entrambi restituiscano dimensioni relative alle stringhe, operano su principi completamente diversi.

	sizeof (Operatore)	strlen (Funzione)
Cosa conta	La dimensione in byte del tipo di dato o dell'array allocato.	Il numero di caratteri prima del terminatore \0.
Terminatore	Include il \0 (se è un array statico).	Esclude il \0.
Quando	Tempo di Compilazione (Statico).	Tempo di Esecuzione (Dinamico).
Libreria	Nessuna (Keyword del linguaggio).	Richiede <cstring>.

3.4.1 Esempio 1: Array Statico (Il caso classico)

Qui sizeof vede l'intero array, mentre strlen conta solo le lettere visibili.

```
1 #include <iostream>
2 #include <cstring>
3
4 char testo[] = "Ciao";
5 // In memoria: ['C', 'i', 'a', 'o', '\0']
6
7 std::cout << strlen(testo); // Output: 4
8 // (C-i-a-o) -> Conta fino a '\0' escluso.
9
10 std::cout << sizeof(testo); // Output: 5
11 // 4 caratteri + 1 terminatore '\0'.
```

3.4.2 Esempio 2: Array con dimensione fissa (Buffer)

Se dichiariamo una dimensione esplicita maggiore del contenuto, la differenza aumenta.

```
1 char buffer[100] = "Hello";
2
3 std::cout << strlen(buffer); // Output: 5
4 // Si ferma dopo la 'o' di Hello.
5
6 std::cout << sizeof(buffer); // Output: 100
7 // Restituisce la dimensione totale allocata dell'array, non importa se
  e' vuoto.
```

3.4.3 Esempio 3: La Trappola del Puntatore (Importante!)

Se usiamo un puntatore invece di un array, sizeof non restituisce più la dimensione della stringa, ma la dimensione del puntatore stesso (indirizzo di memoria).

```
1 const char* ptr = "Esempio molto lungo";
2
3 std::cout << strlen(ptr); // Output: 19 (Corretto, conta i caratteri)
4
5 std::cout << sizeof(ptr); // Output: 8 (su sistemi 64-bit) oppure 4 (su
  32-bit)
```

```

6 // ATTENZIONE: Sta misurando la grandezza della variabile 'ptr',
7 // non della stringa a cui punta!

```

3.5 Input su C-String: Rischi e Soluzioni

L'istruzione `cin >> str;` applicata a un array di `char` (`char str[N]`) è considerata pericolosa e deprecata per l'input utente non controllato.

3.5.1 1. Il Problema del Buffer Overflow

L'operatore `>>` non effettua controlli sui limiti dell'array (bounds checking).

```

1 char str[5];           // Capacita': 4 char + 1 terminatore '\0'
2 cin >> str;           // L'utente digita: "Tantissimo"

```

Effetto in memoria:

- `str[0]..str[4]`: 'T', 'a', 'n', 't', 'i'
- Memoria successiva (FUORI ARRAY): 's', 's', 'i', 'm', 'o', '\0'

Questo sovrascrive variabili adiacenti nello stack, corrompendo lo stato del programma.

3.5.2 2. Il Problema degli Spazi

L'operatore `>>` considera lo spazio come delimitatore. Non è possibile leggere frasi intere.
Input: "Ciao Mondo" → Memorizzato: "Ciao" ("Mondo" resta nel buffer).

3.5.3 3. Le Soluzioni Sicure

A. Soluzione C++ Moderno (Consigliata) Usare `std::string` e `getline`. La stringa si ridimensiona automaticamente.

```

1 #include <string>
2 std::string s;
3 std::getline(std::cin, s); // Legge tutta la riga, dimensione sicura

```

B. Soluzione C-Style (Se obbligati a usare `char[]`) Usare `cin.getline()` specificando la dimensione massima.

```

1 char str[10];
2 // Legge massimo 9 char (lascia spazio per '\0') e si ferma all'invio
3 cin.getline(str, 10);

```

C. Soluzione parziale con `setw` Limita i caratteri letti ma si ferma comunque al primo spazio.

```

1 #include <iomanip>
2 cin >> setw(10) >> str; // Legge max 9 char, sicuro ma niente frasi

```

3.6 Errore: Restituire puntatori a variabili locali

Restituire l'indirizzo di una variabile locale automatica è un errore grave che genera un **Dangling Pointer**.

```
1 int* somma(int a, int b) {  
2     int ris = a + b; // Variabile nello Stack  
3     return &ris;      // ERRORE! 'ris' viene distrutta al return  
4 }
```

Cosa succede: Al ritorno della funzione, lo stack frame viene rimosso ("popped"). Il puntatore restituito punta a una memoria "sporca" o non valida.

3.6.1 Soluzione 1: Usare static (Funzionante ma con limiti)

La variabile statica sopravvive alla fine della funzione.

```
1 int* somma(int a, int b) {  
2     static int ris; // Memoria statica (Global Data)  
3     ris = a + b;  
4     return &ris;    // OK: L'indirizzo resta valido  
5 }  
6 // ATTENZIONE: Ogni chiamata a somma() sovrascrive 'ris' per tutti!
```

3.6.2 Soluzione 2: Allocazione Dinamica (Heap)

Si crea la memoria manualmente. È sicuro ma richiede gestione della memoria.

```
1 int* somma(int a, int b) {  
2     int* ris = new int; // Memoria nello Heap  
3     *ris = a + b;  
4     return ris;  
5 }  
6 // Nota: Il chiamante deve fare 'delete' per evitare Memory Leak.
```

3.6.3 Soluzione 3: Passaggio per Valore (La migliore)

Per tipi semplici come `int`, non ha senso usare puntatori.

```
1 int somma(int a, int b) {  
2     return a + b; // Ritorna una copia del valore. Sicuro e veloce.  
3 }
```

3.7 Ordine di Valutazione e Short-Circuit

3.7.1 1. Il Problema della Divisione per Zero

Nel codice `if (n1 % n2 == 0)`, se l'utente inserisce `n2 = 0`, si verifica un errore di runtime (spesso *Floating Point Exception*) perché la divisione per zero è indefinita.

3.7.2 2. L'Errore Comune (Ordine Sbagliato)

Tentare di correggere il codice aggiungendo il controllo *dopo* l'operazione pericolosa è inutile.

```
1 // ERRORE: Crasha comunque se n2 e' 0  
2 if (n1 % n2 == 0 && n2 != 0) { ... }
```

Motivo: Il C++ valuta le espressioni logiche da **Sinistra a Destra**.

1. Il processore tenta di calcolare `n1 % n2`.
2. Se `n2` è 0, il programma crasha immediatamente.
3. La parte destra (`n2 != 0`) non viene mai raggiunta.

3.7.3 3. La Soluzione Corretta (Sfruttare il Corto Circuito)

Bisogna mettere il controllo di sicurezza **prima** dell'operazione pericolosa.

```
1 // CORRETTO:
2 if (n2 != 0 && n1 % n2 == 0) { ... }
```

Funzionamento del "Short-Circuit Evaluation": In un AND (&&), se la prima condizione è **false**, l'intera espressione è sicuramente falsa. Il compilatore **smette di valutare** il resto per risparmiare tempo.

1. Valuta `n2 != 0`.
2. Se è **false** (cioè `n2` è 0), il programma si ferma e salta l'IF.
3. L'operazione pericolosa `n1 % n2` viene eseguita **solo se** il controllo precedente è passato.

3.8 La Ricorsione

Una funzione è ricorsiva quando invoca se stessa all'interno del proprio corpo.

Struttura Obbligatoria:

1. **Caso Base:** La condizione che termina la catena di chiamate (altrimenti si ha un loop infinito e *Stack Overflow*).
2. **Passo Ricorsivo:** La funzione si richiama su un sotto-problema più piccolo.

```
1 int fattoriale(int n) {
2     // 1. Caso Base: Se n è 1 o 0, il risultato è noto.
3     if (n <= 1) {
4         return 1;
5     }
6
7     // 2. Passo Ricorsivo: n * risultato del problema più piccolo
8     return n * fattoriale(n - 1);
9 }
```

Listing 11: Calcolo del Fattoriale Ricorsivo

3.8.1 Traccia dell'esecuzione (n=3)

Quando chiamiamo `fattoriale(3)`, le chiamate si "impilano" nello Stack (memoria LIFO - Last In, First Out).

1. `fattoriale(3)` viene chiamata. Sospende e chiede: `3 * fattoriale(2)`.
2. `fattoriale(2)` viene chiamata. Sospende e chiede: `2 * fattoriale(1)`.

3. `fattoriale(1)` viene chiamata. **Caso base!** Ritorna 1.
4. `fattoriale(2)` riprende: calcola $2 \times 1 = 2$. Ritorna 2.
5. `fattoriale(3)` riprende: calcola $3 \times 2 = 6$. Ritorna 6.

Vantaggi e Svantaggi:

- (+) Codice molto più pulito ed elegante per problemi matematici o strutture dati come gli Alberi.
- (–) Meno efficiente dei cicli (loop) a causa dell'uso intensivo della memoria Stack. Rischio di *Stack Overflow* se la ricorsione è troppo profonda.

3.9 Controllo del Flusso: Break vs Continue

Queste due istruzioni permettono di alterare il normale flusso di esecuzione all'interno dei cicli (`for`, `while`, `do-while`).

Istruzione	Effetto	Analogia
<code>break</code>	Termina immediatamente il ciclo corrente. L'esecuzione salta alla prima riga fuori dal blocco del ciclo.	"Stop, abbiamo finito. Andiamo a casa."
<code>continue</code>	Salta solo il resto dell'iterazione corrente e forza l'inizio della successiva (dopo l'incremento).	"Questo non mi interessa, passami il prossimo."

```

1 for(int i=0; i < 7; i++) {
2
3     // CASO CONTINUE: Filtro
4     if(i == 2) {
5         continue; // Salta il cout, va a i++ (i diventa 3)
6     }
7
8     // CASO BREAK: Uscita anticipata
9     if(i == 4) {
10        break;    // Uccide il ciclo. Esce totalmente.
11    }
12
13    cout << i << endl;
14 }
15 // Output: 0, 1, 3

```

Listing 12: Esempio Combinato Break/Continue

3.9.1 Nota sui Cicli Annidati (Nested Loops)

L'istruzione `break` esce solo dal ciclo **più interno** in cui si trova.

```

1 for(int i=0; i<3; i++) {
2     for(int j=0; j<3; j++) {
3         if(j==1) break; // Esce solo dal for(j), il for(i) continua!
4     }
5 }

```

3.10 Bitwise Left Shift e Aritmetica

L'operatore di scorrimento a sinistra ($\ll$) è un modo efficiente per moltiplicare interi per potenze di 2.

$$x \ll n \equiv x \times 2^n$$

Nel codice analizzato:

- Input: $a = 1, b = 1, c = 2$.
- Operazione: $a \ll 2$ diventa $1 \times 2^2 = 4$.
- Operazione: $c \ll 2$ diventa $2 \times 2^2 = 8$.
- Somma: $4 + 4 + 8 = 16$.

3.11 Divisione Intera

In C++, il tipo di risultato di un'operazione aritmetica dipende dal tipo degli operandi.

$$\text{int}/\text{int} \rightarrow \text{int}$$

Esempio dell'errore: Dati $a = 1, b = 1, c = 2$.

```
1 int somma = a + b + c; // somma = 4
2 cout << somma / 3;     // Output: 1 (NON 1.33)
```

Il compilatore esegue la divisione intera: $4 : 3 = 1$ con resto di 1. Il resto viene scartato.

3.11.1 Soluzioni per ottenere la media corretta

Per ottenere un risultato decimale, è necessario promuovere l'espressione a `double` o `float`.

Metodo 1: Letterale Double (Il più semplice) Sostituire il numero intero con uno con la virgola.

```
1 // Dividendo per un double, il risultato diventa double
2 cout << (a + b + c) / 3.0; // Output: 1.33333
```

Metodo 2: Casting Esplicito Convertire la somma prima della divisione.

```
1 // Trasformo la somma (int) in double
2 cout << static_cast<double>(a + b + c) / 3;
```

Metodo 3: Cambiare i tipi Dichiarare le variabili come `double` fin dall'inizio.

```
1 double a, b, c;
2 cin >> a >> b >> c;
3 cout << (a + b + c) / 3; // Funziona perché a+b+c è già double
```

3.11.2 Attenzione alla Precedenza degli Operatori

Le parentesi attorno a $(a \ll 2)$ sono fondamentali. In C++, l'operatore somma (+) ha priorità **maggiore** dello shift ($\ll$).

```
1 // CON PARENTESI (Corretto):
2 (a << 2) + (b << 2)
3 // (1 * 4) + (1 * 4) = 4 + 4 = 8
4
5 // SENZA PARENTESI (Errato):
6 a << 2 + b << 2
7 // Il compilatore intende: a << (2 + b) << 2
8 // 1 << (3) << 2
9 // (1 * 8) * 4 = 32! Risultato completamente diverso.
```

3.11.3 Perché usare il casting double()?

L'espressione `double(...)/3` forza la divisione in virgola mobile. Senza il cast, avremmo avuto $16/3 = 5$ (divisione intera con perdita dei decimali).

3.12 Esercizio: Valutazione di Espressioni Logiche

In C++, i valori interi vengono convertiti implicitamente in booleani durante le operazioni logiche:

- $0 \rightarrow \text{false}$
- $n \neq 0 \rightarrow \text{true}$

Analisi del codice:

```
1 int a=3, b=10, c=0;
2
3 // 1. (a && b)
4 // (true && true) -> true -> Stampa 1
5 cout << (a && b) << '\n';
6
7 // 2. (!b || c)
8 // (!true || false) -> (false || false) -> false -> Stampa 0
9 cout << (!b || c) << '\n';
10
11 // 3. (!c && a)
12 // (!false && true) -> (true && true) -> true -> Stampa 1
13 cout << (!c && a) << '\n';
```

3.12.1 Nota: Stampare "true" e "false"

Di default, `cout` stampa i booleani come interi (1 o 0). Per stampare le parole, si usa il manipolatore `boolalpha`.

```
1 cout << boolalpha << (a && b); // Stampa "true"
2 cout << noboolalpha << (a && b); // Torna a stampare "1"
```

3.13 Confronto: Array Dinamico vs Array Statico

La differenza fondamentale sta nel fatto che `sizeof` valuta il **tipo** della variabile, non la memoria puntata.

3.13.1 1. Puntatore allo Heap (new)

```
1 char* str = new char[20];
2 strcpy(str, "hello, world!"); // 13 char
```

- **Memoria:** La variabile `str` è nello Stack (8 byte), l'array di 20 char è nello Heap.
- `strlen(str)`: **13**. Legge il contenuto nello Heap.
- `sizeof(str)`: **8**. Misura la grandezza del puntatore stesso. **Non vede** i 20 byte allocati.

3.13.2 2. Array nello Stack

```
1 char str[20];
2 strcpy(str, "hello, world!");
```

- **Memoria:** Tutto l'array (20 byte) risiede nello Stack.
- `strlen(str)`: **13**.
- `sizeof(str)`: **20**. Il compilatore conosce la dimensione totale dell'array.

Dichiarazione	strlen	sizeof	Note
char* p = new char[20]	13	8	Perde l'info sulla dimensione
char a[20]	13	20	Mantiene l'info sulla dimensione

Ricorda: Se passi un array statico a una funzione, esso decade in un puntatore. Dentro la funzione, `sizeof` restituirà 8, non la dimensione originale!

3.14 1. Puntatore a Dati Costanti (Il caso richiesto)

Sintassi: `const int* p`; oppure `int const* p`; (sono identici).

Qui la protezione è sul **contenuto**. Il puntatore è libero di muoversi, ma non ha i permessi di scrittura.

```
1 int x = 10;
2 int y = 20;
3 const int* p = &x; // p punta a x
4
5 *p = 15; // ERRORE! Il valore è "read-only" tramite p.
6 p = &y; // OK! Posso cambiare il bersaglio del puntatore.
```

Utilizzo tipico: Passare parametri a funzioni per lettura (es. `print(const char* str)`) per garantire che la funzione non modifichi i dati originali.

3.15 2. Puntatore Costante (Costante all'Indirizzo)

Sintassi: `int* const p;`

Qui la protezione è sull'**indirizzo**. Il puntatore nasce e muore puntando alla stessa variabile, ma può modificarne il contenuto.

```
1 int x = 10;
2 int y = 20;
3 int* const p = &x; // p è "sposato" con x
4
5 *p = 15; // OK! Posso modificare il valore di x.
6 p = &y; // ERRORE! Non posso cambiare l'indirizzo.
```

Nota: Deve essere inizializzato immediatamente alla dichiarazione.

3.16 3. Costante Totale

Sintassi: `const int* const p;`

Non si può cambiare né il valore, né l'indirizzo. È una "foto" immutabile di una specifica cella di memoria.

```
1 const int* const p = &x;
2 // *p = 15; // ERRORE
3 // p = &y; // ERRORE
```

3.16.1 Tabella Riassuntiva

Dichiarazione	Cambiare Indirizzo (p = ...)	Cambiare Valore (*p = ...)
<code>const int* p</code>	SI	NO
<code>int* const p</code>	NO	SI
<code>const int* const p</code>	NO	NO

4 Scope, Shadowing e Tempo di Vita

4.1 Il Fenomeno del Shadowing (Mascheramento)

Quando una variabile locale ha lo stesso nome di una variabile globale, la locale ha la precedenza all'interno del suo blocco. La variabile globale viene "oscurata".

```
1 #include <iostream>
2 using namespace std;
3
4 int x = 5; // Variabile Globale
5
6 void f() {
7     int x = 7; // Variabile Locale (nasconde la globale)
8
9     cout << x; // Output: 7 (usa la locale)
10    cout << ::x; // Output: 5 (usa la globale grazie a ::)
11 }
```

L'operatore :: (Scope Resolution Operator) Preposto al nome di una variabile senza specificare un namespace, forza il compilatore a cercare la variabile nello scope globale (Global Scope).

4.2 Confronto dei Tempi di Vita (Lifetime)

È fondamentale distinguere tra Scope (dove la variabile si "vede") e Lifetime (quando la variabile "esiste" in memoria).

Tipo Variabile	Tempo di Vita (Lifetime)	Allocazione in Memoria
Globale	Static Duration. Nasce all'avvio del programma e viene distrutta solo alla fine del main.	<i>Data Segment</i> (Memoria Statica).
Locale	Automatic Duration. Nasce quando l'esecuzione entra nel blocco <code>{ }</code> e viene distrutta appena si esce <code>}</code> .	<i>Stack</i> (Pila).

4.3 Conversione Implicita Intero-Booleano

In C++, le condizioni degli `if` e dei cicli accettano espressioni aritmetiche. Il risultato numerico viene convertito in booleano secondo la regola:

$$0 \rightarrow \text{false}$$

$$\text{Qualsiasi altro numero}(\pm 1, 100, -5) \rightarrow \text{true}$$

Dati: $a = 1, b = 0, c = 3$.

4.3.1 Analisi Caso 1: Valutazione Completa

```
1 if ((a+b) && (2*b))
```

1. Valuta $(a+b) \rightarrow 1 + 0 = 1$. È `true`.
2. L'operatore `&&` richiede di valutare il secondo operando.
3. Valuta $(2*b) \rightarrow 2 \times 0 = 0$. È `false`.
4. `true && false` → **Risultato: false**.

4.3.2 Analisi Caso 2: Cortocircuito

```
1 if ((2*b) && (a+c))
```

1. Valuta $(2*b) \rightarrow 0$. È `false`.
2. Poiché il primo operando di un AND è falso, il risultato è già deciso.
3. $(a+c)$ viene **ignorato** (Short-Circuit).
4. **Risultato: false**.

Nota bene: Questo codice compila senza errori, ma l'uso di calcoli matematici complessi dentro un `if` riduce la leggibilità. È meglio scrivere esplicitamente `if ( (a+b) != 0 ... )`.

4.4 Precedenza degli Operatori ed Errori di Runtime

Esercizio: Calcolo Espressioni Miste In C++, gli operatori moltiplicativi (*, /, %) hanno priorità maggiore rispetto a quelli additivi (+, -).

4.4.1 Caso 1: L'errore del Modulo Zero

```
1 int c = 3*-2 + 4%0 + 10/3;
```

Questo codice provoca un **Crash a Runtime**. L'operazione $4 \% 0$ (resto della divisione per zero) è illegale quanto la divisione per zero. Il processore segnala un'eccezione e il programma termina.

4.4.2 Caso 2: Calcolo Passo-Passo

```
1 int c = 3*-2 + 4%3 + 10/3;
```

L'espressione viene risolta raggruppando le operazioni ad alta priorità:

$$c = (3 \times -2) + (4\%3) + (10/3)$$

1. $3 * -2 \rightarrow -6$.
2. $4 \% 3 \rightarrow$ Il 3 sta nel 4 una volta col resto di **1**.
3. $10 / 3 \rightarrow$ Divisione intera. 3.33... viene troncato a **3**.
4. **Somma Finale:** $-6 + 1 + 3 = -2$.

Risultato: c vale **-2**.

4.5 Errore Comune: Il Casting "Tardivo"

In C++, il tipo di un'operazione è determinato **solo** dai tipi degli operandi, non dal tipo della variabile che riceverà il risultato o dal cast esterno.

4.5.1 Analisi dell'Errore

```
1 double x = 10 / 3;           // x vale 3.0
2 double y = double(10/3);    // y vale 3.0
```

Passaggi eseguiti dalla CPU:

1. **Valutazione Interna:** 10 e 3 sono int. Viene eseguita la divisione intera.

$$10 \div 3 = 3 \quad (\text{parte decimale persa})$$

2. **Promozione/Casting:** Il risultato intero 3 viene convertito in double.

$$3 \rightarrow 3.0$$

4.5.2 La Soluzione Corretta

Per ottenere la divisione con la virgola, bisogna convertire **almeno uno** degli operandi **prima** che avvenga la divisione.

```
1 // Metodo 1: Casting di un operando
2 double a = double(10) / 3; // 10.0 / 3 -> 3.333...
3
4 // Metodo 2: Letterale con virgola
5 double b = 10.0 / 3; // 10.0 / 3 -> 3.333...
6
7 // Metodo 3: Moltiplicazione furba (Attenzione alla precedenza!)
8 double c = 1.0 * 10 / 3; // (1.0 * 10) diventa double, poi diviso 3
```

4.6 I Puntatori in C++

Un puntatore è una variabile destinata a contenere un indirizzo di memoria. Ogni puntatore ha un tipo specifico (es. `int*`, `char*`) che indica al compilatore come interpretare i dati trovati a quell'indirizzo.

```
1 int x = 10; // Variabile intera (valore: 10)
2 int* p; // Dichiarazione di un puntatore a intero
3 p = &x; // Assegnazione: p ora contiene l'indirizzo di x
```

Simbolo	Nome	Funzione
&	Operatore di Indirizzo	Restituisce l'indirizzo di memoria di una variabile (&x).
*	Operatore di Dereferenziazione	Accede al valore contenuto all'indirizzo puntato (*p).

4.7 Esempio di Utilizzo

Modificare il valore di una variabile tramite il suo puntatore (Indirezione).

```
1 int a = 5;
2 int* ptr = &a; // ptr punta ad a
3
4 cout << ptr; // Stampa un indirizzo (es. 0x7ffee...)
5 cout << *ptr; // Stampa il valore puntato (5)
6
7 *ptr = 20; // Modifica il valore all'indirizzo di a
8 cout << a; // Stampa 20 (a è cambiata!)
```

4.8 Il Puntatore Nullo (nullptr)

È buona norma inizializzare sempre i puntatori. Se non puntano a nulla di valido, usare `nullptr` (in C++11) o `NULL`.

```
1 int* p = nullptr; // Sicuro. Non punta a memoria casuale.
```

4.9 Dichiarazione e Inizializzazione dei Puntatori

4.9.1 Sintassi di Base

La dichiarazione avviene antepoendo l'asterisco al nome della variabile o posponendolo al tipo.

```
1 int* p;    // Stile C++ (consigliato: evidenzia il tipo)
2 int *p;    // Stile C (valido)
3 int * p;   // Valido ma sconsigliato
```

4.9.2 Inizializzazione

A differenza delle *Reference*, i puntatori **non devono** essere inizializzati obbligatoriamente alla dichiarazione, ma lasciarli non inizializzati crea "Puntatori Selvaggi" (Wild Pointers).

```
1 // PERICOLOSO (Wild Pointer)
2 int* p;
3 // p contiene un indirizzo casuale (es. 0x12F4...).
4 // Scrivere *p = 10; ora causerebbe un CRASH sicuro.
5
6 // SICURO (Inizializzazione a Null)
7 int* p = nullptr; // Standard C++11 (meglio di NULL)
8 // Ora p non punta a nulla. È facile controllare se è valido:
9 if (p != nullptr) { ... }
10
11 // SICURO (Inizializzazione a Variabile)
12 int x = 5;
13 int* p = &x;
```

4.9.3 L'Errore della Dichiarazione Multipla

L'asterisco vale solo per la singola variabile successiva.

```
1 int* a, b;
2 // a è un puntatore a int (int*)
3 // b è un normale intero (int)
4
5 // Corretto:
6 int *a, *b;
```

5 Aritmetica dei Puntatori e Matrici

5.1 1. Il concetto di "Passo" (Stride)

Dato un puntatore $T^* p$ e un intero N :

$$\text{Indirizzo}(p + N) = \text{Indirizzo}(p) + (N \times \text{sizeof}(T))$$

5.2 2. Equivalenza Array-Puntatore

Per un array `int arr[5]`, le seguenti espressioni sono identiche:

- `arr[3]`
- `*(arr + 3)`

5.3 3. Linearizzazione delle Matrici (Row-Major)

Una matrice statica `T mat[R][C]` è memorizzata come un unico blocco contiguo. L'elemento alla riga i e colonna j si trova all'indirizzo:

$$\text{Addr}(i, j) = \text{Base} + ((i \times C + j) \times \text{sizeof}(T))$$

Dove C è il numero totale di colonne (la larghezza della riga).

5.3.1 Accesso tramite Puntatori (La formula esplicita)

L'espressione `mat[i][j]` viene tradotta dal compilatore come:

$$*(*(\text{mat} + i) + j)$$

Analisi passo-passo:

1. `mat`: Puntatore alla prima riga (tipo `int (*)[C]`).
2. `mat + i`: Sposta il puntatore avanti di i righe (ogni passo è $C \times 4$ byte).
3. `*(mat + i)`: Dereferenzia la riga. Otteniamo il puntatore base della riga i .
4. `... + j`: Sposta il puntatore avanti di j interi.
5. `* (...)`: Legge il valore finale.

6 Gestione dei File in C++

Per lavorare con i file, includiamo la libreria `<fstream>`. I file vengono trattati come flussi di dati (stream), analogamente a `cin` e `cout`.

6.1 1. Scrittura su File (ofstream)

La classe `ofstream` (Output File Stream) si usa per scrivere.

```
1 #include <fstream>
2 // ...
3 ofstream fileOut("output.txt"); // Apre (o crea) il file
4
5 if (!fileOut.is_open()) {
6     cerr << "Errore creazione file!" << endl;
7 } else {
8     fileOut << "Ciao Mondo!" << endl; // Scrittura
9     fileOut << 42 << endl;           // Scrittura numeri
10    fileOut.close();                 // Buona norma chiudere sempre
11 }
```

Modalità di Apertura: Di default, `ofstream` sovrascrive il file. Per aggiungere dati in coda (Append), si usa il flag `ios::app`:

```
1 ofstream file("log.txt", ios::app); // Non cancella il contenuto
   precedente
```

6.2 2. Lettura da File (ifstream)

La classe `ifstream` (Input File Stream) si usa per leggere.

- L'operatore `>>` legge parola per parola (salta gli spazi), come `cin`.
- La funzione `getline()` legge intere righe.

```
1 ifstream fileIn("dati.txt");
2
3 if (!fileIn) { // Controllo rapido se il file esiste
4     cerr << "File non trovato!" << endl;
5     return 1;
6 }
7
8 string parola;
9 while (fileIn >> parola) {
10     // Legge finche' ci sono parole nel file
11     cout << "Letto: " << parola << endl;
12 }
13 fileIn.close();
```

6.3 3. Gestione degli Errori e Stati dello Stream

Quando si lavora con i file, possono verificarsi vari errori (file inesistente, disco pieno, formato sbagliato, fine del file). Ogni oggetto stream mantiene dei "flag" di stato:

Funzione	Significato
<code>good()</code>	Tutto ok. Nessun bit di errore attivo.
<code>eof()</code>	End Of File . Siamo arrivati alla fine del file.
<code>fail()</code>	Errore logico (es. cerco di leggere un <code>int</code> ma trovo "ciao").
<code>bad()</code>	Errore grave di I/O (es. disco rotto).

6.3.1 Il Loop di Lettura Sicuro

Il modo standard per leggere tutto un file controllando gli errori è inserire l'operazione di lettura direttamente nella condizione del `while`.

```
1 int numero;
2 ifstream f("numeri.txt");
3
4 // Questo ciclo fa due cose:
5 // 1. Prova a leggere un numero
6 // 2. Restituisce true se la lettura ha avuto successo, false se finisce
   // il file o c'e' un errore
7 while (f >> numero) {
8     cout << numero << " ";
9 }
10
11 // Verifica post-lettura: perche' siamo usciti dal while?
12 if (f.eof()) {
13     cout << "\nLettura completata con successo (Fine File).";
14 } else if (f.fail()) {
15     cout << "\nErrore di formato nel file (es. lettere al posto di
       numeri).";
16 }
```


7 Gestione della Memoria Dinamica (Heap)

La memoria dinamica permette di allocare dati la cui dimensione o durata non è nota a tempo di compilazione.

7.1 Operatori new e delete

7.1.1 1. Singola Variabile

```
1 // Allocazione
2 int* p = new int;           // Alloca un intero (valore indefinito)
3 int* p2 = new int(5);       // Alloca e inizializza a 5
4
5 // Utilizzo
6 *p = 10;
7
8 // Deallocazione
9 delete p; // Restituisce la memoria
10 p = nullptr; // Evita Dangling Pointer
```

7.1.2 2. Array Dinamici

Attenzione: per gli array bisogna usare le parentesi quadre anche nella delete.

```
1 int size = 10;
2 // Allocazione
3 int* arr = new int[size];
4
5 // Deallocazione
6 delete[] arr; // Le parentesi [] sono OBBLIGATORIE!
```

Nota: Se si usa `delete arr` (senza quadre) su un array, si ha Undefined Behavior (spesso libera solo il primo elemento).

7.2 Problemi Comuni

1. **Memory Leak (Perdita di Memoria):** Si verifica quando si perde il puntatore a un'area di memoria allocata senza aver fatto delete. Quella memoria rimane occupata (e inutilizzabile) fino alla chiusura del programma.

```
1 void leak() {
2     int* p = new int[100];
3     // Manca delete[] p;
4     // Uscendo dalla funzione, la variabile 'p' (nello stack)
5     muore,
6     // ma i 100 interi (nello heap) restano orfani.
7 }
```

2. **Dangling Pointer (Puntatore Pendente):** Si verifica quando si usa un puntatore dopo aver fatto delete.

```
1 delete p;
2 *p = 5; // ERRORE GRAVE! Accesso a memoria non più tua.
3
```

8 Parametri Formali vs Parametri Attuali

È fondamentale distinguere tra la definizione della funzione e la sua chiamata.

8.1 Definizioni

- **Parametri Formali:** Sono le variabili elencate nella definizione della funzione. Agiscono come variabili locali all'interno della funzione e servono da "contenitori" per i valori in ingresso.
- **Parametri Attuali (Argomenti):** Sono i valori (o le variabili) reali che vengono passati alla funzione nel momento in cui viene chiamata nel `main` o altrove.

8.2 Esempio Pratico

```
1 // Definizione della funzione
2 // 'a' e 'b' sono PARAMETRI FORMALI
3 int somma(int a, int b) {
4     return a + b;
5 }
6
7 int main() {
8     int x = 10;
9     int y = 5;
10
11     // Chiamata della funzione
12     // 'x' e 'y' sono PARAMETRI ATTUALI (variabili)
13     int ris1 = somma(x, y);
14
15     // Chiamata con letterali
16     // '100' e '20' sono PARAMETRI ATTUALI (valori costanti)
17     int ris2 = somma(100, 20);
18 }
```

8.3 Tabella di Confronto

Caratteristica	Parametri Formali	Parametri Attuali
Dove sono?	Nella firma della funzione	Nella chiamata della funzione
Ruolo	Variabili locali (Segnaposto)	Valori da elaborare
Esistenza	Solo durante l'esecuzione della funzione	Esistono nello scope del chiamante

9 Passaggio Parametri: Valore vs Riferimento

In C++, possiamo decidere come i dati vengono trasferiti alle funzioni.

9.1 1. Passaggio per Valore (Pass-by-Value)

È il metodo di default. I parametri formali sono variabili locali distinte che vengono inizializzate con una **copia** dei valori dei parametri attuali.

```

1 void scambioFinto(int a, int b) {
2     int temp = a;
3     a = b;
4     b = temp;
5     // Qui a e b sono scambiati, ma sono solo COPIE locali.
6 }
7
8 int main() {
9     int x = 10, y = 20;
10    scambioFinto(x, y);
11    // x vale ancora 10, y vale ancora 20. Nulla è cambiato.
12 }

```

9.2 2. Passaggio per Riferimento (Pass-by-Reference)

Utilizzando il simbolo & dopo il tipo del parametro, creiamo un *alias*. Il parametro formale diventa un "soprannome" per la variabile originale. Non avviene nessuna copia.

```

1 // Notare la '&' vicino a int
2 void scambioVero(int &a, int &b) {
3     int temp = a;
4     a = b;      // Modifica direttamente la memoria del chiamante
5     b = temp;
6 }
7
8 int main() {
9     int x = 10, y = 20;
10    scambioVero(x, y);
11    // Ora x vale 20 e y vale 10. Lo scambio è avvenuto.
12 }

```

9.3 Tabella Comparativa

	Per Valore	Per Riferimento (&)
Sintassi	void f(int x)	void f(int &x)
Memoria	Viene creata una copia nello Stack della funzione.	Non si crea nuova memoria dati, solo un alias.
Effetti Collaterali	Nessuno. L'originale è protetto.	Le modifiche si ripercuotono sull'originale.
Efficienza	Bassa per oggetti grandi (struct, vector).	Alta (costo costante, come un puntatore).

10 Classi di Memorizzazione (Storage Classes)

Le storage classes determinano il tempo di vita (lifetime), la visibilità (scope) e il collegamento (linkage) delle variabili.

10.1 1. Variabili Automatiche (Default)

Le variabili dichiarate dentro un blocco { } senza keyword specifiche sono automatiche.

- **Keyword storica:** `auto` (ora usata per type inference).
- **Memoria:** Stack.
- **Vita:** Temporanea (Scope del blocco).

10.2 2. La keyword `static`

Il comportamento cambia in base al contesto:

10.2.1 A. Static Locale (dentro una funzione)

Preserva il valore tra diverse chiamate della funzione. Viene inizializzata **una sola volta** alla prima esecuzione.

```

1 void contatore() {
2     static int c = 0; // Eseguita solo la prima volta
3     c++;
4     cout << c << " ";
5 }
6
7 int main() {
8     contatore(); // Stampa 1
9     contatore(); // Stampa 2 (si ricorda di c!)
10    contatore(); // Stampa 3
11 }

```

Memoria: Data Segment (non Stack!).

10.2.2 B. Static Globale (fuori dalle funzioni)

Limita la visibilità della variabile al solo file corrente (*Internal Linkage*). Utile per incapsulare variabili di supporto che non devono essere viste da altri file `.cpp`.

10.3 3. La keyword `extern`

Dichiara una variabile (o funzione) che è definita in un altro file sorgente (*External Linkage*).

```

1 // File1.cpp
2 int globale = 100;
3
4 // File2.cpp
5 extern int globale; // Non crea nuova memoria, punta a quella di File1
6 void stampa() { cout << globale; }

```

10.4 4. La keyword `mutable`

Utilizzabile solo su membri di una classe. Permette di modificare il membro anche se l'oggetto è `const`.

```

1 class Esempio {
2     mutable int accessi;
3 public:
4     void getDato() const {
5         accessi++; // OK anche se il metodo è const!

```

```

6     }
7 };

```

10.5 Tabella Riassuntiva

Classe	Keyword	Memoria	Tempo di Vita
Automatica	(nessuna)	Stack	Blocco di codice
Statica	<code>static</code>	Data Segment	Intero programma
Registro	<code>register</code>	Registro CPU	Blocco (Deprecato in C++17)
Esterna	<code>extern</code>	Data Segment	Intero programma

A differenza delle storage classes standard (`static`, `extern`), la memoria dinamica non è attivata da una keyword nella dichiarazione della variabile, ma dall'uso esplicito di operatori di allocazione.

10.6 Distinzione Puntatore vs Oggetto

È cruciale distinguere tra la variabile che "tiene" l'indirizzo e l'oggetto puntato.

```

1 void funzione() {
2     // p è una variabile AUTOMATICA (Stack).
3     // L'oggetto creato da 'new' è DINAMICO (Heap).
4     int* p = new int(10);
5
6     // ...
7
8     delete p;
9     // Ora l'oggetto nello Heap è distrutto.
10    // La variabile p sullo Stack esiste ancora (ma è un dangling
11    // pointer)
12    // finché non finisce la funzione.
13 }

```

10.7 Tabella Completa delle 4 Durate (C++ Standard)

Tipo di Durata	Come si ottiene	Chi decide la morte?
1. Automatic	Variabili locali (default)	Il compilatore (alla chiusura della }).
2. Static	Globali o <code>static</code> locali	Il compilatore (alla fine del programma).
3. Dynamic	Operatore <code>new</code>	Il programmatore (quando chiama <code>delete</code>).
4. Thread	Keyword <code>thread_local</code>	Il sistema (alla fine del thread).

11 L-value e R-value

11.1 Definizioni Fondamentali

- **L-value (Locator Value):** Un oggetto che ha un indirizzo di memoria persistente. Può stare a sinistra o a destra dell'assegnazione. È il "contenitore".
- **R-value (Read/Right Value):** Un valore temporaneo che non ha un indirizzo accessibile. Può stare solo a destra dell'assegnazione. È il "contenuto".

11.2 1. Prefisso e PostFisso, Differenza Semantica

La differenza sta nell'ordine in cui avviene l'incremento rispetto alla valutazione dell'espressione.

- **Prefisso (++x):** Incrementa la variabile **prima** di restituire il valore. (Incrementa → Usa).
- **Postfisso (x++):** Restituisce il valore **corrente** (copia) e incrementa **dopo**. (Usa → Incrementa).

11.3 2. Esempio di Codice

```
1 int main() {  
2     // Caso 1: PREFISSO  
3     int x = 5;  
4     int y = ++x; // x diventa 6, poi y prende 6  
5     // Risultato: x = 6, y = 6  
6  
7     // Caso 2: POSTFISSO  
8     int a = 5;  
9     int b = a++; // b prende 5 (valore vecchio), poi a diventa 6  
10    // Risultato: a = 6, b = 5  
11 }
```

11.4 3. Performance e Best Practice

Per i tipi primitivi (`int`), non c'è differenza di velocità. Tuttavia, per **oggetti complessi** (come gli iteratori STL), il prefisso è più efficiente.

- `++it`: Modifica l'oggetto direttamente.
- `it++`: Deve creare una **copia temporanea** del vecchio valore per poterlo restituire, spreca risorse.

Consiglio: Nei cicli `for`, preferire sempre `++i`.

12 Operatori Logici e Short-Circuit

12.1 1. Tabella degli Operatori

Servono a combinare espressioni booleane. In C++, 0 è `false`, qualsiasi altro numero è `true`.

- **AND (&&):** Restituisce `true` solo se **tutti** gli operandi sono veri.
- **OR (||):** Restituisce `true` se **almeno uno** degli operandi è vero.
- **NOT (! operator):** Operatore unario. Inverte il valore di verità (`!true` → `false`).

12.2 2. Precedenza degli Operatori

L'ordine di esecuzione, dal più prioritario al meno prioritario, è:

1. `!` (NOT) - Vince su tutti.
2. `&&` (AND) - Viene valutato prima dell'OR.
3. `||` (OR) - Ha la priorità più bassa.

Esempio: `A || B && C` viene letto come `A || (B && C)`.

12.3 3. Short-Circuit Evaluation (Corto Circuito)

Il C++ interrompe la valutazione dell'espressione appena il risultato è certo. Questo comportamento è fondamentale per scrivere codice sicuro.

```
1 int main() {
2     int* ptr = nullptr;
3
4     // SICURO: La seconda condizione (*ptr == 10) NON viene eseguita
5     // perché la prima (ptr != nullptr) è falsa.
6     // L'AND si ferma subito.
7     if (ptr != nullptr && *ptr == 10) {
8         // ...
9     }
10
11     int x = 10;
12     // EFFICIENTE: Se x == 10 è vero, la funzione calcoloPesante()
13     // NON viene chiamata, perché l'OR è già soddisfatto.
14     if (x == 10 || calcoloPesante()) {
15         // ...
16     }
17 }
```

13 Matrici (Array Multidimensionali)

13.1 1. Definizione e Dichiarazione

Una matrice è concettualmente una tabella bidimensionale, ma tecnicamente è un "array di array". La sintassi richiede di specificare prima le righe e poi le colonne.

```

1 // Dichiarazione di una matrice 3x4 (3 Righe, 4 Colonne)
2 int mat[3][4] = {
3     {1, 2, 3, 4},    // Riga 0
4     {5, 6, 7, 8},    // Riga 1
5     {9, 10, 11, 12} // Riga 2
6 };

```

13.2 2. Accesso agli elementi

L'accesso avviene tramite doppio indice: `mat[riga][colonna]`. Gli indici partono sempre da 0.

- `mat[0][0]`: Primo elemento (in alto a sinistra).
- `mat[2][3]`: Ultimo elemento (in basso a destra nell'esempio sopra).

13.3 3. Iterazione (Cicli Annidati)

Per scorrere una matrice si usano comunemente due cicli `for` annidati: quello esterno per le righe, quello interno per le colonne.

```

1 // Stampa della matrice
2 for (int i = 0; i < 3; i++) {           // Scorre le righe
3     for (int j = 0; j < 4; j++) {       // Scorre le colonne
4         cout << mat[i][j] << " ";
5     }
6     cout << endl; // A capo alla fine di ogni riga
7 }

```

13.4 4. Layout in Memoria (Row-Major)

In C++, le matrici sono memorizzate in modo contiguo "per righe". L'elemento `mat[0][3]` è immediatamente seguito in memoria da `mat[1][0]`. Non esistono "buchi" tra la fine di una riga e l'inizio della successiva.

14 La Grammatica del Linguaggio

14.1 1. Definizione

La grammatica di un linguaggio di programmazione è un insieme formale di regole che descrive come combinare i simboli (token) per creare istruzioni valide. Si divide principalmente in tre livelli:

- **Analisi Lessicale:** Riconosce le "parole" valide (keyword come `int`, operatori come `++`, identificatori).
- **Sintassi:** Definisce l'ordine corretto dei token (es. le parentesi devono essere bilanciate, il punto e virgola va alla fine).
- **Semantica:** Definisce il significato delle istruzioni (es. il controllo dei tipi).

14.2 2. Sintassi vs Semantica

È fondamentale distinguere tra la forma e il significato.

```
1 // Errore di SINTASSI (Syntax Error)
2 // Viola le regole strutturali (manca il punto e virgola, ordine errato)
3
4 int x = 10 // Manca ;
5 if (x > 5 // Manca )
6
7 // Errore SEMANTICO (Semantic Error)
8 // La struttura è corretta, ma l'operazione è illogica o non permessa.
9 int a = "ciao"; // Assegnazione di tipo incompatibile
10 int b = 10 / 0; // Divisione per zero (runtime o compile time warning)
```

15 Numeri Reali (Floating Point)

15.1 1. Standard IEEE 754

I numeri reali sono rappresentati in virgola mobile, simile alla notazione scientifica:

$$\text{Valore} = (-1)^{\text{Segno}} \times (1.\text{Mantissa}) \times 2^{(\text{Esponente}-\text{Bias})}$$

La memoria viene divisa in tre parti:

1. **Segno (S):** 1 bit (0 positivo, 1 negativo).
2. **Esponente (E):** Determina l'ordine di grandezza (dove sta la virgola).
3. **Mantissa (M):** Contiene le cifre significative (precisione).

15.2 2. Tipi comuni

- **float (32 bit):** 1 bit Segno, 8 bit Esponente, 23 bit Mantissa. Precisione: 7 cifre decimali.
- **double (64 bit):** 1 bit Segno, 11 bit Esponente, 52 bit Mantissa. Precisione: 15 cifre decimali.

15.3 3. Problemi di Approssimazione

Alcuni numeri decimali non sono rappresentabili esattamente in binario. **Regola d'oro:** Mai confrontare due float con ==. Usare una soglia di tolleranza (ϵ).

16 Tipi Enumerati (enum)

16.1 1. Definizione

L'enum permette di definire un nuovo tipo di dato che assume un insieme ristretto di valori costanti, associando etichette leggibili a numeri interi (partendo da 0).

16.2 2. Enum Classico (C-Style)

I valori sono visibili globalmente (rischio di conflitti).

```
1 enum Colore {
2     ROSSO,    // Vale 0
3     VERDE,    // Vale 1
4     BLU       // Vale 2
5 };
6
7 Colore c = ROSSO;
8 // int x = ROSSO; // Valido (conversione implicita a int)
```

16.3 3. Enum Class (C++11 - Strongly Typed)

Più sicuro. I valori non vengono convertiti implicitamente in interi e devono essere prefissati dal nome dell'enum.

```
1 enum class Stato {
2     INIZIO,
3     GIOCO,
4     GAME_OVER
5 };
6
7 // Stato s = INIZIO; // ERRORE!
8 Stato s = Stato::INIZIO; // CORRETTO
9
10 // if (s == 0) // ERRORE! (Non è un int)
```

17 Dimensione e Range dei Tipi Primitivi

17.1 1. Tabella delle dimensioni (Architettura 64-bit tipica)

Sebbene lo standard definisca solo le relazioni minime, questa è la situazione più comune su sistemi moderni (Windows/Linux 64-bit).

Tipo	Byte	Range Approssimativo
bool	1	true (1) o false (0)
char	1	-128 a 127 (ASCII)
short	2	± 32.768 ($\approx \pm 3 \times 10^4$)
int	4	$\pm 2.147.483.647$ ($\approx \pm 2 \times 10^9$)
long long	8	$\pm 9 \times 10^{18}$ (Enorme)
float	4	$\pm 3.4 \times 10^{38}$ (7 cifre precisione)
double	8	$\pm 1.7 \times 10^{308}$ (15 cifre precisione)

17.2 2. Signed vs Unsigned

Aggiungendo la keyword `unsigned`, si elimina il segno negativo e si raddoppia il limite positivo.

$$Range_{signed} = [-2^{n-1}, 2^{n-1} - 1]$$

$$Range_{unsigned} = [0, 2^n - 1]$$

Esempio con `int` (32 bit):

- **signed int:** da -2 Miliardi a $+2$ Miliardi.
- **unsigned int:** da 0 a $+4$ Miliardi.

17.3 3. Ottenere i limiti via codice

Non imparare i numeri a memoria! Usa la libreria `<limits>`.

```

1 #include <iostream>
2 #include <limits> // Libreria fondamentale
3 using namespace std;
4
5 int main() {
6     cout << "Dimensione int: " << sizeof(int) << " byte" << endl;
7
8     // Trova il valore massimo possibile per un int
9     cout << "Max Int: " << numeric_limits<int>::max() << endl;
10
11    // Trova il valore minimo possibile per un int
12    cout << "Min Int: " << numeric_limits<int>::min() << endl;
13
14    // Esempio Unsigned
15    cout << "Max Unsigned Int: " << numeric_limits<unsigned int>::max()
16    << endl;

```

18 Linkage Interno ed Esterno

18.1 1. Concetto di Unità di Traduzione

Il "Linkage" determina se una variabile o funzione definita in un file sorgente (.cpp) è visibile e accessibile da altri file sorgente durante la fase di linking.

18.2 2. Internal Linkage (Visibilità Locale)

Il simbolo è visibile **solo all'interno del file** in cui è dichiarato. Altri file non possono accedervi. Se un altro file dichiara una variabile con lo stesso nome, sono due entità distinte in memoria.

Si ottiene con:

- Keyword `static` (su variabili globali o funzioni libere).
- **Namespace Anonimi** (Standard C++ moderno).
- Variabili `const` globali (default in C++).

```

1 // File: A.cpp
2 static int segreto = 10; // Visibile solo in A.cpp
3
4 void f() {
5     segreto++; // OK
6 }

```

18.3 3. External Linkage (Visibilità Globale)

Il simbolo è unico per l'intero programma. Tutti i file vedono la **stessa** locazione di memoria.

Si ottiene con:

- Variabili globali non statiche.
- Keyword `extern` (usata per dichiarare l'esistenza della variabile in altri file).

```
1 // File: Globals.cpp
2 int punteggio = 0; // Definizione (alloca memoria)
3
4 // File: Main.cpp
5 extern int punteggio; // Dichiarazione (dice: "esiste altrove")
6
7 int main() {
8     punteggio = 50; // Modifica la variabile in Globals.cpp
9 }
```

18.4 4. Errore Comune: One Definition Rule (ODR)

Se definisci una variabile con external linkage in un file Header (.h), e questo header viene incluso in due .cpp diversi, il Linker darà errore ("Multiple Definition"), perché proverà a creare due variabili globali con lo stesso nome visibili a tutti. **Soluzione:** Nel .h metti solo `extern`, la definizione va in un solo .cpp. Una delle trappole più comuni del C++ riguarda il linkage predefinito delle variabili globali.

18.5 1. La regola generale (Variabili non-const)

Le variabili globali normali hanno, di default, **External Linkage**. Se scrivi `int x = 10;` nel global scope, quella `x` è potenzialmente visibile da tutto il programma.

18.6 2. L'eccezione del C++ (Variabili const)

In C++, le variabili dichiarate `const` nel global scope hanno di default **Internal Linkage**. Questo comportamento è diverso dal C (dove sono External).

```
1 // File: Data.cpp
2 const int MAX_VITE = 3;
3 // In C++ equivale a: static const int MAX_VITE = 3;
4 // È visibile SOLO dentro Data.cpp.
```

18.7 3. Come forzare una const a essere External

Se vuoi definire una costante globale condivisa tra tutti i file (es. in un file `Constants.cpp` e usata ovunque), devi usare esplicitamente `extern` sia nella definizione che nella dichiarazione.

```
1 // File: Constants.cpp (Definizione)
2 extern const double GRAVITA = 9.81; // "extern" forza il linkage esterno
3
4 // File: Constants.h (Dichiarazione per gli altri)
```

```

5 extern const double GRAVITA;
6
7 // File: Main.cpp
8 #include "Constants.h"
9 // Ora GRAVITA è visibile qui.

```

19 L'istruzione typedef e gli Alias

19.1 1. Definizione

La keyword `typedef` (Type Definition) serve a creare un **alias** (soprannome) per un tipo di dato esistente. Non crea un nuovo tipo fisico in memoria, ma introduce un nuovo identificatore per riferirsi a esso.

Sintassi: `typedef <tipo_esistente> <nuovo_nome>;`

19.2 2. Esempi di utilizzo

19.2.1 A. Semplificazione di tipi lunghi

```

1 typedef unsigned long long int uint64;
2
3 int main() {
4     uint64 numero_gigante = 10000000000; // Molto più breve da scrivere
5 }

```

19.2.2 B. Chiarezza Semantica

Aiuta a capire cosa rappresenta una variabile, anche se il tipo sottostante è banale.

```

1 typedef float Prezzo;
2 typedef float Peso;
3
4 Prezzo costo = 9.99;
5 Peso quantita = 2.5;
6 // Per il compilatore sono entrambi float, ma per l'uomo è chiaro.

```

19.2.3 C. Puntatori (e sicurezza)

Usare `typedef` per i puntatori evita errori comuni di dichiarazione.

```

1 typedef int* ptr_int;
2
3 ptr_int a, b;
4 // Entrambi sono puntatori (int* a, int* b).
5
6 // SENZA typedef:
7 int* x, y;
8 // ATTENZIONE: x è un puntatore, ma y è solo un intero normale!

```

19.3 3. typedef vs #define

- **#define:** Sostituzione testuale gestita dal preprocessore. Non rispetta le regole di scope (visibilità) e può causare errori logici coi puntatori.
- **typedef:** Istruzione interpretata dal compilatore. Rispetta lo scope ed è "type-safe".

19.4 4. C++ Moderno: la keyword using

Dallo standard C++11, si preferisce usare **using** per una sintassi più chiara (e compatibile coi template).

```
1 // Vecchio stile
2 typedef int* IntPtr;
3
4 // Nuovo stile (Consigliato)
5 using IntPtr = int*;
```

20 Il Metalinguaggio

20.1 1. Definizione Concettuale

Un metalinguaggio è un linguaggio formale utilizzato per descrivere, analizzare o definire le regole di un altro linguaggio (chiamato **Linguaggio Oggetto**).

Metalinguaggio $\xrightarrow{\text{descrivere}}$ Linguaggio Oggetto

20.2 2. Esempi nel mondo reale

- **Linguistica:** Un libro di grammatica inglese scritto in italiano. (Italiano = Metalinguaggio, Inglese = Linguaggio Oggetto).
- **Informatica (Sintassi):** La **BNF** (Backus-Naur Form) è il metalinguaggio usato per definire la grammatica del C++.
- **Informatica (Dati):** L'**XML** è un metalinguaggio che permette di definire altri linguaggi di markup (come SVG o XHTML).

20.3 3. Esempio pratico: BNF

La BNF usa simboli propri (come $::=$, $<$, $>$, $|$) per spiegare al compilatore come è fatto un numero intero.

```
1 // Questo codice NON è C++, è BNF (Metalinguaggio)
2 // Definisce cosa sia una "cifra" e un "intero"
3
4 <digit> ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
5 <integer> ::= <digit> | <digit> <integer>
6
7 // Grazie a queste regole, il C++ (Linguaggio Oggetto)
8 // capisce che "42" è un numero valido.
```

21 Classificazione dei Tipi di Dato

21.1 1. Tipi Predefiniti (Fondamentali)

Sono i tipi "nativi" integrati nel linguaggio. Il compilatore conosce intrinsecamente la loro dimensione e le operazioni aritmetiche applicabili.

- **Aritmetici:** Interi (`int`, `short`, `long`, `char`) e Virgola Mobile (`float`, `double`).
- **Booleani:** `bool` (Vero/Falso).
- **Void:** `void` (Assenza di tipo/valore).
- **Nullptr:** `std::nullptr_t` (C++11).

21.2 2. Tipi Derivati (Composti)

Sono costruiti a partire dai tipi fondamentali (o da altri tipi derivati). La loro esistenza dipende dal tipo di base a cui si riferiscono.

```
1 int x = 10;           // Tipo Predefinito (int)
2
3 // --- TIPI DERIVATI ---
4 int lista[5];         // Array di int
5 int* ptr = &x;        // Puntatore a int
6 int& ref = x;         // Riferimento a int
7 int funzione();      // Funzione che ritorna int
```

21.3 3. Tipi Definiti dall'Utente (UDT)

Sono aggregati complessi definiti dal programmatore per modellare oggetti reali. Una volta definiti, si comportano come nuovi tipi.

```
1 // Struttura (C-style) o Classe (C++)
2 struct Studente {
3     char nome[50];    // Derivato (Array)
4     int matricola;    // Predefinito
5 };
6
7 Studente s1; // s1 è di tipo "Studente"
```

21.4 4. Nota Bene: Le Stringhe

In C++, `std::string` **non** è un tipo primitivo, ma una classe della libreria standard che gestisce internamente un array di `char`.

22 I Literals (Costanti Letterali)

22.1 1. Definizione

Un literal è un valore fisso inserito direttamente nel codice sorgente. Non è il risultato di un calcolo né il contenuto di una variabile, ma il dato grezzo stesso.

```

1 int a = 10;           // 10 è un Integer Literal
2 double b = 3.14;     // 3.14 è un Floating-point Literal
3 char c = 'A';        // 'A' è un Character Literal
4 bool d = true;       // true è un Boolean Literal

```

22.2 2. Tipi di Literal e Prefissi (Basi Numeriche)

Per i numeri interi, possiamo indicare la base numerica usando dei prefissi:

- **Decimale (Base 10):** Nessun prefisso (es. 42).
- **Ottale (Base 8):** Inizia con 0 (es. 052 → 42 dec). *Attenzione a non mettere zeri iniziali per sbaglio!*
- **Esadecimale (Base 16):** Inizia con 0x (es. 0x2A → 42 dec).
- **Binario (Base 2):** Inizia con 0b (C++14, es. 0b101010 → 42 dec).

22.3 3. Suffissi per forzare il tipo

Di default, un numero intero è `int` e un decimale è `double`. Possiamo cambiare questo comportamento con i suffissi.

Suffisso	Tipo Risultante	Esempio
U o u	unsigned int	10u
L o l	long	100L
LL	long long	100LL
F o f	float	3.14f
L (su float)	long double	3.14L

22.4 4. Literal Stringa e Carattere

- **Char:** Singoli apici. Tipo `char`. Es: `'A'`.
- **String:** Doppi apici. Tipo `const char[]`. Es: `"Ciao"`.
- **Escape Sequences:** Caratteri speciali che iniziano con backslash.
 - `'\n'` (Nuova riga)
 - `'\t'` (Tabulazione)
 - `'\0'` (Terminatore stringa - Null character)

23 Conversione Implicita (Coercion)

23.1 1. Definizione

La conversione implicita avviene quando il compilatore trasforma automaticamente il tipo di un'espressione in un altro per permettere un'operazione o un'assegnazione.

23.2 2. Promozione (Widening - Sicura)

Si verifica quando si converte un tipo più piccolo in uno più grande (o più preciso). Non c'è perdita di informazione. La gerarchia tipica è:

`bool → char → short → int → long → float → double`

```
1 int a = 10;
2 double b = a; // Sicuro: a diventa 10.0
```

23.3 3. Restringimento (Narrowing - Rischiosa)

Si verifica quando si converte un tipo grande in uno più piccolo. Questo causa la **perdita di dati**.

```
1 double pi = 3.14159;
2 int x = pi;
3 // x diventa 3. La parte decimale viene TRONCATA (non arrotondata).
4
5 int grande = 1000;
6 char piccolo = grande;
7 // Overflow: il valore 1000 non sta in un char (max 127).
8 // Risultato imprevedibile (solitamente modulo 256).
```

23.4 4. Conversioni Aritmetiche Usuali

Quando un operatore agisce su operandi di tipi diversi, il tipo "inferiore" viene promosso al tipo "superiore" prima del calcolo.

```
1 int i = 5;
2 double d = 2.5;
3
4 // i viene promosso a double (5.0) temporaneamente
5 double risultato = i + d; // 5.0 + 2.5 = 7.5
```

23.5 5. Il caso del bool

- Da numero a bool: 0 diventa `false`, qualsiasi altro numero (anche negativo) diventa `true`.
- Da bool a numero: `false` diventa 0, `true` diventa 1.

24 Strumenti per Programmare: Il Toolchain

24.1 1. Componenti Fondamentali

Per trasformare il codice sorgente in un programma funzionante, servono tre passaggi logici gestiti da strumenti diversi:

1. **Editor:** Scrittura del file sorgente (`.cpp`).
2. **Compilatore:** Traduzione del sorgente in linguaggio macchina intermedio (*Codice Oggetto*, file `.obj` o `.o`). Verifica la sintassi.

3. **Linker:** Unisce il codice oggetto del nostro programma con le librerie esterne (es. `iostream`) per creare l'unico file **Eseguibile** (`.exe`).

24.2 2. IDE (Integrated Development Environment)

Un IDE è un software che racchiude tutti questi strumenti in un'unica interfaccia grafica, permettendo di scrivere, compilare e correggere errori (debug) nello stesso posto.

- **Esempi:** Visual Studio (Microsoft), CLion (JetBrains), Xcode (Apple).
- **VS Code:** È un *Text Editor* avanzato che diventa simile a un IDE tramite estensioni.

24.3 3. Il Processo di Build (Compilazione)

Quando premiamo "Compila ed Esegui", avvengono 3 fasi distinte:

- **Fase 1: Preprocessing**
Il preprocessore risolve le direttive che iniziano con `#` (es. `#include`). Sostituisce il testo delle librerie nel file sorgente.
- **Fase 2: Compilazione (Compilation)**
Il codice viene analizzato sintatticamente e tradotto in *Assembly* e poi in codice oggetto binario. Se c'è un errore di sintassi, il processo si ferma qui.
- **Fase 3: Linking**
Il Linker collega i file oggetto tra loro. Se manca una definizione (es. chiami una funzione che non esiste), si ottiene un *Linker Error*.

25 Il Linker (Collegatore)

25.1 1. Definizione e Scopo

Il Linker è l'ultimo strumento della catena di build. Il suo compito è combinare uno o più file oggetto (generati dal compilatore) e le librerie di sistema in un unico file eseguibile (o libreria condivisa).

25.2 2. Input e Output

File Oggetto (`.obj`) + Librerie Statiche (`.lib`) $\xrightarrow{\text{LINKER}}$ Eseguibile (`.exe`)

25.3 3. Funzioni Principali

1. **Risoluzione dei Simboli (Symbol Resolution):** Associa ogni riferimento a una funzione o variabile (es. una chiamata di funzione) alla sua effettiva definizione in memoria.
2. **Rilocazione (Relocation):** Il compilatore genera indirizzi relativi (partendo da 0 per ogni file). Il Linker unisce i file e ricalcola gli indirizzi di memoria corretti affinché le parti del programma non si sovrappongano.

25.4 4. Errori Tipici (Linker Errors)

Gli errori di linking avvengono **dopo** che la compilazione ha avuto successo.

- **Undefined Reference / Unresolved External Symbol:** Hai dichiarato una funzione (header file) e l'hai usata, ma non hai mai scritto la sua implementazione (corpo) nel file .cpp, oppure non hai incluso quel file .cpp nel progetto.
- **Multiple Definition:** Hai definito la stessa variabile globale o funzione (non inline) in due file diversi, oppure in un header file incluso più volte senza protezioni.

26 Approccio Compilato vs Interpretato

26.1 1. Linguaggi Compilati (es. C++)

Il codice sorgente viene tradotto interamente in linguaggio macchina (binario) da un programma chiamato **Compilatore** in una fase separata, prima dell'esecuzione.

- **Prestazioni:** Elevatissime. La CPU esegue istruzioni native ottimizzate.
- **Portabilità:** Bassa. Un eseguibile compilato per Windows non gira su Linux. Bisogna ricompilare per ogni sistema.
- **Rilevamento Errori:** Molti errori (sintassi e tipi) vengono trovati subito, durante la compilazione (Compile-time).
- **Privacy:** Il codice sorgente non viene distribuito, solo il binario illeggibile.

26.2 2. Linguaggi Interpretati (es. Python)

Non esiste una fase di compilazione preliminare. Un software chiamato **Interprete** legge, analizza ed esegue il codice sorgente riga per riga durante l'esecuzione stessa.

- **Prestazioni:** Più lente (overhead della traduzione in tempo reale).
- **Portabilità:** Alta. Lo stesso codice sorgente gira ovunque sia installato l'interprete.
- **Rilevamento Errori:** Gli errori si scoprono solo quando il programma ci "passa sopra" (Runtime), rischiando crash improvvisi.
- **Sviluppo:** Ciclo di modifica-test molto rapido (non serve ricompilare).

26.3 3. Tabella Riassuntiva

Caratteristica	Compilato (C++)	Interpretato (Python)
Traduzione	Tutto in una volta (prima)	Istruzione per istruzione (durante)
Velocità CPU	Massima (nativo)	Ridotta (mediata)
Portabilità	Richiede ricompilazione	Immediata (basta l'interprete)
Distribuzione	File eseguibile (.exe)	Codice sorgente (.py)

26.4 1. Errori di Compilazione (Syntax Errors)

Vengono rilevati dal compilatore prima che il programma possa essere eseguito. Impediscono la generazione del file eseguibile.

- **Natura:** Violazione delle regole grammaticali del linguaggio.
- **Facilità di risoluzione:** Alta (il compilatore indica la riga).

```
1 int main() {  
2     int x = 10 // ERRORE: Manca il punto e virgola  
3     if (x = 5) // WARNING: Assegnazione invece di confronto (==)  
4 }
```

26.5 2. Errori di Runtime (Execution Errors)

Si verificano mentre il programma è in esecuzione, causando un arresto anomalo (Crash) o un comportamento imprevisto.

- **Natura:** Operazioni illegali su dati o memoria.
- **Esempi classici:** Divisione per zero, Dereferenziazione di puntatori nulli, Stack Overflow (ricorsione infinita).

```
1 int a = 10;  
2 int b = 0;  
3 int c = a / b; // CRASH: Divisione per intero zero (Floating point  
    exception)
```

26.6 3. Errori Logici (Logic Errors / Bugs)

Il programma compila e non crasha, ma produce un risultato errato. Sono i più difficili da individuare (richiedono testing e debugging manuale).

- **Natura:** Errore nel ragionamento o nell'algoritmo.

```
1 // Voglio calcolare la media di due numeri  
2 double media = 10 + 20 / 2;  
3 // RISULTATO: 20 (Perché la divisione ha precedenza!)  
4 // CORRETTO: (10 + 20) / 2 = 15
```

27 Programmazione Modulare

27.1 1. Definizione

La programmazione modulare è una tecnica di progettazione che suddivide un programma complesso in componenti separati, indipendenti e riutilizzabili chiamati **moduli**. L'obiettivo è il "Divide et Impera": gestire la complessità spezzettando il problema.

27.2 2. Struttura dei File in C++

In C++, un modulo è solitamente diviso in due file distinti:

- **File Header (.h) - L'Interfaccia:** Contiene le **dichiarazioni** (funzioni, classi, costanti). Rappresenta il "Menu": dice all'esterno *cosa* fa il modulo, ma non come.
- **File Source (.cpp) - L'Implementazione:** Contiene le **definizioni** (il codice effettivo tra parentesi graffe). Rappresenta la "Cucina": contiene la logica interna nascosta all'utente.

27.3 3. Header Guards (Guardie di inclusione)

Sono direttive del preprocessore obbligatorie nei file .h. Servono a impedire che lo stesso file venga incluso più volte nello stesso sorgente, causando errori di *Multiple Definition*.

```
1 #ifndef MIO_MODULO_H // Se non è definito...
2 #define MIO_MODULO_H // ...definiscilo ora.
3
4 // --- Qui vanno le dichiarazioni ---
5 void saluta();
6 int somma(int a, int b);
7
8 #endif // Fine del blocco
```

27.4 4. Vantaggi

- **Manutenibilità:** I bug sono isolati in file specifici.
- **Compilazione:** Se cambi un modulo, ricompili solo quello e non tutto il progetto.
- **Riutilizzo:** Puoi prendere il file .h e .cpp e portarli in un altro progetto.

28 Operatori di Shift (Scorrimento)

28.1 1. Concetto Generale

Gli operatori di shift spostano la rappresentazione binaria di un numero intero verso sinistra o destra di un numero specificato di posizioni. Sono molto efficienti e corrispondono a moltiplicazioni o divisioni per potenze di 2.

28.2 2. Left Shift (◀)

Sposta i bit a sinistra. Le posizioni vacanti a destra vengono riempite con 0.

$$x \ll n \iff x \times 2^n$$

```
1 int a = 5; // Binario: 0000 0101
2 int b = a << 2; // Sposto a sx di 2
3 // Risultato: 0001 0100 (Decimale: 20)
4 // Calcolo: 5 * 2^2 = 5 * 4 = 20
```

28.3 3. Right Shift (»)

Sposta i bit a destra. I bit meno significativi vengono persi.

$$x \gg n \iff x/2^n \quad (\text{Divisione Intera})$$

Comportamento sul riempimento a sinistra:

- **Unsigned (Logical Shift):** Entrano sempre 0.
- **Signed (Arithmetic Shift):** Solitamente viene replicato il **Bit di Segno** (il bit più a sinistra) per mantenere il numero positivo o negativo.

```
1 int a = 40;          // Binario: ...0010 1000
2 int b = a >> 3;      // Sposto a dx di 3
3 // Risultato: ...0000 0101 (Decimale: 5)
4 // Calcolo: 40 / 2^3 = 40 / 8 = 5
```

29 Differenza tra #define, const e inline

29.1 1. Costanti: #define vs const

In C++, è preferibile usare `const` rispetto a `#define`.

Caratteristica	#define (C style)	const (C++ style)
Natura	Sostituzione testo (Preprocessore)	Variabile sola lettura (Compilatore)
Controllo Tipi	Nessuno (Type-unsafe)	Forte (Type-safe)
Scope	Globale (ignora { })	Rispetta lo scope (locale/namespace)
Debugging	Invisibile (è solo un numero)	Visibile (ha un nome e indirizzo)

29.2 2. Funzioni: Macro vs Inline

Con `#define` si possono definire macro parametriche che sembrano funzioni, ma agiscono tramite sostituzione del testo.

```
1 // MACRO (Pericolosa)
2 #define MAX(A, B) ((A) > (B) ? (A) : (B))
3
4 // FUNZIONE INLINE (Sicura)
5 inline int max(int a, int b) {
6     return (a > b) ? a : b;
7 }
```

29.3 3. Il pericolo dei Side Effects nelle Macro

Le macro duplicano il testo degli argomenti. Se l'argomento contiene un'operazione (es. `x++`), questa viene eseguita più volte.

```
1 #define QUADRATO(x) ((x) * (x))
2
3 int a = 3;
4 int ris = QUADRATO(a++);
5 // Espansione: ((a++) * (a++))
6 // Risultato: a viene incrementato DUE volte! Errore logico.
```

30 Scope, Namespace e Visibilità

30.1 1. Scope (Ambito)

Lo scope definisce la "vita" e la "visibilità" di una variabile. È delimitato dalle parentesi graffe { }.

- **Scope Locale:** Variabili dichiarate dentro una funzione. Non sono accessibili dall'esterno.
- **Scope Globale:** Variabili dichiarate fuori da tutto. Visibili ovunque.
- **Shadowing (Mascheramento):** Se una variabile locale ha lo stesso nome di una globale, la locale ha la precedenza e "nasconde" quella globale.

30.2 2. Namespace (Spazio dei nomi)

Un namespace è un contenitore logico per raggruppare identificatori (nomi di funzioni, classi, variabili) e prevenire conflitti di nome (Name Collisions). È come il "cognome" delle funzioni.

```
1 namespace Matematica {
2     int somma(int a, int b) { return a + b; }
3 }
4
5 namespace Stringhe {
6     int somma(int a, int b) { return 0; } // Stesso nome, nessun
7     conflitto
8 }
9 int x = Matematica::somma(1, 2);
```

30.3 3. Operatore di Risoluzione di Scope (::)

L'operatore :: (Scope Resolution Operator) serve per specificare a quale ambito appartiene un nome.

- **Uso standard:** Namespace::Funzione (es. std::cout).
- **Uso unario (Global Scope):** ::variabile. Serve per accedere a una variabile globale se questa è stata "nascosta" da una locale con lo stesso nome.

```
1 int x = 10; // Variabile Globale
2
3 int main() {
4     int x = 5; // Variabile Locale (nasconde la globale)
5
6     // cout << x;    -> Stampa 5 (Locale)
7     // cout << ::x;  -> Stampa 10 (Globale)
8 }
```

30.4 4. La direttiva using

Permette di usare i nomi di un namespace senza dover digitare ogni volta il prefisso.

- **Using Declaration (Specifica):** Importa solo un nome. È la pratica consigliata.
`using std::cout;` → Puoi scrivere `cout` invece di `std::cout`.
- **Using Directive (Totale):** Importa TUTTO il namespace. È rischiosa (Namespace Pollution).
`using namespace std;`

Nota Importante: Mai scrivere `using namespace` dentro un file header (`.h`)! Forzerebbe chiunque includa quel file a importare tutto il namespace, creando caos.

31 Streams e Manipolazione dell'Input/Output

31.1 1. Definizione di Stream

Uno stream è un'astrazione software che rappresenta un flusso sequenziale di dati (byte) da una sorgente (Input) o verso una destinazione (Output), indipendentemente dal dispositivo fisico (tastiera, file, monitor).

31.2 2. Gli Oggetti Standard (<iostream>)

- **std::cin (Character Input):** Flusso di ingresso standard (solitamente la tastiera).
- **std::cout (Character Output):** Flusso di uscita standard (solitamente il monitor).
- **std::cerr:** Flusso di errore (non bufferizzato, appare subito).

31.3 3. Manipolazione del Formato (<iomanip>)

I manipolatori sono comandi inseriti nel flusso tramite l'operatore « per modificare la visualizzazione dei dati.

A. Manipolatori di Base (senza parametri):

```
1 cout << hex << 42; // Stampa "2a" (cambia la base in Hex)
2 cout << dec << 42; // Torna a Decimale "42"
3 cout << left;      // Allinea il testo a sinistra
```

B. Manipolatori con Parametri (richiedono <iomanip>):

```
1 #include <iomanip>
2
3 // setw(n): Imposta la larghezza del campo (SOLLO per il prossimo dato)
4 cout << setw(10) << "Ciao"; // Stampa "          Ciao"
5
6 // setfill(c): Cambia il carattere di riempimento
7 cout << setfill('*') << setw(5) << 10; // Stampa "***10"
8
9 // setprecision(n): Imposta il numero di cifre/decimali
10 double pi = 3.141592;
11 cout << fixed << setprecision(2) << pi; // Stampa "3.14"
```


31.4 4. Persistenza dei Manipolatori (Sticky vs Non-Sticky)

- **Sticky (Persistenti):** Mantengono il loro effetto su tutti i dati successivi finché non vengono annullati esplicitamente.
Esempi: `hex`, `dec`, `setprecision`, `setfill`.
- **Non-Sticky (Temporanei):** L'effetto vale solo per l'operazione di output immediatamente successiva.
Esempio: `setw(n)`. Bisogna riscriverlo per ogni variabile se si vuole incolonnare una tabella.

32 Overloading degli Operatori di Input/Output

32.1 1. L'Obiettivo

Permettere l'uso diretto degli oggetti delle nostre classi con `std::cout` e `std::cin`, esattamente come se fossero tipi primitivi.

32.2 2. La Regola Fondamentale

L'operatore « (o ») non può essere una funzione membro della classe.

- **Motivo:** L'operando di sinistra è lo stream (`ostream` o `istream`), non il nostro oggetto.
- **Soluzione:** Si definisce come funzione globale esterna. Spesso si dichiara **friend** dentro la classe per accedere ai membri `private`.

32.3 3. Sintassi e Pattern Standard

È fondamentale restituire il riferimento allo stream (`ostream&`) per permettere il "Chaining" (concatenazione) delle chiamate (es. `cout << obj << endl;`).

```
1 #include <iostream>
2 using namespace std;
3
4 class Punto {
5 private:
6     int x, y;
7
8 public:
9     Punto(int x, int y) : x(x), y(y) {}
10
11     // Dichiarazione FRIEND: permette alla funzione di leggere x e y
12     // privati
13     friend ostream& operator<<(ostream& os, const Punto& p);
14     friend istream& operator>>(istream& is, Punto& p);
15 };
16
17 // --- IMPLEMENTAZIONE (Fuori dalla classe) ---
18
19 // Output (Stampa)
20 ostream& operator<<(ostream& os, const Punto& p) {
21     // Non uso cout, ma 'os' passato come parametro!
```

```

21     os << "(" << p.x << ", " << p.y << ")";
22     return os; // Restituisco lo stream per concatenare
23 }
24
25 // Input (Lettura)
26 istream& operator>>(istream& is, Punto& p) {
27     // Non uso cin, ma 'is'
28     is >> p.x >> p.y;
29     return is;
30 }
31
32 // --- UTILIZZO NEL MAIN ---
33 /*
34 Punto p1(0,0);
35 cin >> p1;          // Ora funziona!
36 cout << p1 << endl; // Ora funziona!
37 */

```

32.4 4. Punti Chiave

1. **Return Type:** Deve essere `ostream&` (per riferimento), mai `ostream` (per valore), perché gli stream non si possono copiare.
2. **Parametri:**
 - Lo stream è sempre per riferimento (`&`) perché viene modificato.
 - L'oggetto in output è `const` (sola lettura).
 - L'oggetto in input **non** è `const` (deve essere riempito).

33 C-Strings: strlen, sizeof e Input

Consideriamo la seguente inizializzazione:

```

1 char str[] = "ciao mondo!\n";

```

33.1 1. Layout in Memoria

Il compilatore aggiunge automaticamente il terminatore nullo `\0`.

c	i	a	o		s	p		m	o	n	d	o	!	\n	\0
---	---	---	---	--	---	---	--	---	---	---	---	---	---	----	----

33.2 2. strlen vs sizeof

- **strlen(str):** Restituisce la lunghezza **logica** della stringa. Conta i caratteri fino al terminatore (escluso).
Calcolo: 4 (ciao) + 1 (spazio) + 6 (mondo!) + 1 (newline) = **12**.
- **sizeof(str):** Restituisce la dimensione **fisica** dell'array in byte. Include il terminatore nullo implicito.
Calcolo: 12 (caratteri) + 1 (terminatore) = **13**.

33.3 3. Il problema del cin (Doppio Input)

L'operatore di estrazione » interpreta lo spazio bianco come un "fine lettura".

```
1 char a[50], b[50];
2 // Utente scrive: "ciao mondo"
3
4 cin >> a;
5 // a diventa "ciao". Il cursore si ferma allo spazio.
6
7 cin >> b;
8 // b diventa "mondo".
9 // Non attende input dall'utente, legge cio che e' rimasto nel buffer!
```

Soluzione per stringhe con spazi: Usare la funzione `cin.getline()`:

```
1 cin.getline(a, 50); // Legge tutta la riga inclusi gli spazi
```

34 Problemi di Buffer e Soluzioni

34.1 1. Il Buffer Condiviso

Il buffer di input (`stdin`) è una coda unica. Se un'operazione di lettura non consuma tutto l'input (es. si ferma a uno spazio), i dati rimanenti restano nel buffer e verranno letti automaticamente dal prossimo `cin`, causando comportamenti imprevisti (salti di input).

34.2 2. Soluzione: `cin.ignore()`

Per "pulire" il buffer dagli avanzzi indesiderati prima di una nuova lettura, si usa la funzione `ignore`.

```
1 #include <iostream>
2 #include <limits> // Necessario per numeric_limits
3
4 using namespace std;
5
6 int main() {
7     int eta;
8     char nome[50];
9
10    cout << "Inserisci eta': ";
11    cin >> eta;
12    // Se l'utente scrive "25 anni", il cin prende 25.
13    // " anni\n" rimane nel buffer!
14
15    // PULIZIA DEL BUFFER
16    // Ignora fino al carattere 'a capo' (\n) o per massimo 1000
17    // caratteri
18    cin.ignore(numeric_limits<streamsize>::max(), '\n');
19
20    cout << "Inserisci nome: ";
21    cin.getline(nome, 50);
22    // Ora funziona! Senza ignore, getline avrebbe letto subito
23    // lo spazio o l'invio rimasto prima, salvando una stringa vuota.
24 }
```

34.3 3. La Formula Magica

La riga standard per pulire completamente il buffer è:

```
cin.ignore(numeric_limits<streamsize>::max(), 'n');
```

Significa: "Ignora tutto quello che c'è nel tubo finché non trovi un Invio (così ripartiamo da una riga pulita)".

35 Overloading dell'Operatore di Assegnamento (operator=)

35.1 1. Perché ridefinirlo?

Se una classe gestisce memoria dinamica (puntatori), l'assegnamento di default effettua una **Shallow Copy** (copia solo l'indirizzo del puntatore). Questo causa due problemi gravi:

1. **Memory Leak:** La memoria puntata dall'oggetto di sinistra viene persa (nessuno la dealloca).
2. **Double Free:** Alla distruzione, entrambi gli oggetti tenteranno di fare `delete` sulla stessa area di memoria.

Occorre quindi implementare una **Deep Copy**.

35.2 2. Lo Schema dei 4 Passaggi (The Pattern)

Per implementare correttamente `operator=`, bisogna seguire rigorosamente questi step per garantire la sicurezza (specialmente contro l'auto-assegnazione).

```
1 class Vettore {
2 private:
3     int* dati;
4     int size;
5
6 public:
7     // ... costruttori ...
8
9     // Operatore di Assegnamento
10    Vettore& operator=(const Vettore& altro) {
11
12        // 1. CHECK AUTO-ASSEGNAZIONE (Identity Test)
13        // Se sto assegnando l'oggetto a se stesso (v = v), non faccio
14        // nulla.
15        // Fondamentale: senza questo, lo step 2 distruggerebbe la
16        // sorgente!
17        if (this == &altro) {
18            return *this;
19        }
20
21        // 2. PULIZIA (Cleanup)
22        // Libero la memoria che questo oggetto possedeva prima.
23        delete[] dati;
```

```

22
23     // 3. DEEP COPY (Riallocazione e Copia)
24     // Prendo le dimensioni dell'altro e alloco nuova memoria
25     size = altro.size;
26     dati = new int[size];
27
28     // Copio i valori uno per uno
29     for (int i = 0; i < size; i++) {
30         dati[i] = altro.dati[i];
31     }
32
33     // 4. RESTITUIRE *THIS
34     // Permette la concatenazione (a = b = c)
35     return *this;
36 }
37 };

```

35.3 3. Regola

Se la tua classe ha bisogno di definire esplicitamente uno tra:

- Distruttore
- Costruttore di Copia
- Operatore di Assegnamento

Allora quasi sicuramente **ha bisogno di tutti e tre**.

36 NULL vs nullptr

36.1 1. Il problema di NULL

Nelle vecchie versioni di C++, NULL è una macro che si espande allo zero intero (0). Questo crea ambiguità nell'overloading delle funzioni, poiché il compilatore lo interpreta come un `int` e non come un puntatore.

```

1 void f(int x);      // Versione 1
2 void f(char* p);   // Versione 2
3
4 f(NULL); // CHIAMA LA VERSIONE 1 (int)!
5 // Perché NULL == 0. Errore semantico.

```

36.2 2. La soluzione: nullptr

Introdotta in C++11, `nullptr` è una parola chiave che rappresenta un puntatore nullo con un proprio tipo specifico (`std::nullptr_t`).

- Si converte implicitamente in qualsiasi tipo di puntatore.
- **Non** si converte implicitamente in un intero.

```

1 f(nullptr); // CHIAMA LA VERSIONE 2 (char*). Corretto.

```

36.3 3. Best Practice

Utilizzare sempre `nullptr` in C++ moderno. Usare `NULL` (o `0`) per i puntatori è considerato deprecato e pericoloso (legacy code).

37 La Keyword friend

37.1 1. Definizione

Il meccanismo `friend` permette a una funzione esterna o a un'altra classe di accedere ai membri **private** e **protected** della classe che concede l'amicizia. Rappresenta una deroga all'incapsulamento.

37.2 2. Sintassi

La dichiarazione `friend` deve essere inserita all'interno della classe che vuole condividere i suoi dati.

```
1 class Scatola {
2 private:
3     int segreto;
4
5 public:
6     Scatola(int s) : segreto(s) {}
7
8     // Dichiaro che la funzione 'apri' e' amica.
9     // Può trovarsi sotto public, private o protected, non cambia nulla
10    .
11    friend void apri(Scatola& s);
12 };
13 // Funzione GLOBALE (non membro)
14 void apri(Scatola& s) {
15     // Accesso consentito a 'segreto' grazie a friend
16     cout << "Il segreto e': " << s.segreto << endl;
17 }
```

37.3 3. Caratteristiche Fondamentali (Regole)

- **Asimmetrica:** Se A dichiara B friend, B accede ad A, ma A non accede a B.
- **Intransitiva:** Se A è friend di B, e B è friend di C, A non ha privilegi su C.
- **Non Ereditata:** Le classi derivate non ereditano gli amici della classe base, né l'amicizia della classe base si estende automaticamente alle derivate.

37.4 4. Quando usarla?

- **Overloading operatori I/O:** (`operator<<`, `operator>>`) che devono essere funzioni globali ma necessitano accesso ai dati interni.
- **Testing:** Per permettere alle unit test di verificare lo stato interno privato.

- **Pattern specifici:** Come il pattern *Memento* o classi Contenitore/Iteratore.

Nota: L'uso eccessivo di `friend` è sconsigliato perché rompe l'incapsulamento (Information Hiding). Va usato solo quando strettamente necessario.

38 Lista di Inizializzazione (Member Initialization List)

38.1 1. Sintassi

È situata dopo i parametri del costruttore, preceduta da `:` e separata da virgole.

```
1 class Esempio {
2     int x;
3     string s;
4 public:
5     // Costruttore con Lista di Inizializzazione
6     Esempio(int val) : x(val), s("Ciao") {
7         // Qui dentro x ed s esistono già' e hanno i loro valori.
8         // Il corpo {} puo' rimanere vuoto.
9     }
10};
```

38.2 2. Differenza con l'Assegnamento

- **Assegnamento (nel corpo):** L'oggetto viene prima costruito (default) e poi modificato. È un doppio lavoro (Costruzione + Assegnazione).
- **Inizializzazione (nella lista):** L'oggetto viene costruito direttamente con il valore finale. È più performante (Solo Costruzione).

38.3 3. Quando è OBBLIGATORIA?

In questi casi l'assegnamento nel corpo genera errore di compilazione:

A. Membri const

```
1 class Test {
2     const int maxVal;
3 public:
4     // Test(int v) { maxVal = v; } <- ERRORE: non puoi modificare const
5     !
6     Test(int v) : maxVal(v) {} // OK: inizializzato alla nascita
7};
```

B. Membri Reference (&)

```
1 class Test {
2     int& ref;
3 public:
4     // I reference devono essere legati subito a qualcosa.
5     Test(int& target) : ref(target) {}
6};
```

C. Oggetti membri senza costruttore di default Se la classe contiene un oggetto che richiede obbligatoriamente parametri, bisogna fornirli nella lista.

D. Chiamata al costruttore della Classe Base Nell'ereditarietà, per passare parametri al padre.

```
1 Derivata(int x) : Base(x) { ... }
```

38.4 4. Ordine di Inizializzazione (Trappola)

I membri vengono inizializzati **nell'ordine in cui sono dichiarati nella classe**, NON nell'ordine in cui li scrivi nella lista.

```
1 class Trappola {
2     int a;
3     int b;
4 public:
5     // ERRORE LOGICO! 'a' viene inizializzato prima di 'b'.
6     // Qui 'b' contiene spazzatura quando viene usato per inizializzare
7     // 'a'.
8     Trappola(int val) : b(val), a(b) {}
9 };
```

39 Il Puntatore this

39.1 1. Definizione

Il puntatore `this` è un puntatore speciale, accessibile solo all'interno delle funzioni membro non statiche di una classe. Esso punta all'indirizzo dell'oggetto che ha invocato la funzione.

- **Tipo:** `NomeClasse* const this` (è un puntatore costante, non puoi fargli puntare un altro oggetto).
- **Esistenza:** Viene passato implicitamente dal compilatore come argomento nascosto.

39.2 2. Principali Utilizzi

A. Risoluzione di ambiguità (Shadowing) Quando i nomi dei parametri coincidono con i nomi dei membri.

```
1 class Punto {
2     int x, y;
3 public:
4     Punto(int x, int y) {
5         // x = x;      <- Ambiguo (assegna parametro a parametro)
6         this->x = x; // <- Chiaro: Membro = Parametro
7         this->y = y;
8     }
9 };
```

B. Restituire l'oggetto corrente (Concatenazione) Restituendo `*this` (l'oggetto dereferenziato) come *reference*, si possono concatenare le chiamate (Fluent Interface).

```
1 class Calcolatrice {
2     int val;
3 public:
4     Calcolatrice& aggiungi(int n) {
5         val += n;
6         return *this; // Restituisce l'oggetto stesso
7     }
8 };
```



```

7     }
8
9     Calcolatrice& toglia(int n) {
10         val -= n;
11         return *this;
12     };
13 };
14
15 // Utilizzo:
16 calc.aggiungi(10).toglia(3).aggiungi(5);

```

39.3 3. La Limitazione dei Metodi Statici

Le funzioni dichiarate `static` appartengono alla classe, non a una specifica istanza. Pertanto, al loro interno **il puntatore `this` non esiste**. Tentare di usare `this` in un metodo statico genera errore di compilazione.

40 Puntatori a Oggetti e Layout di Memoria

40.1 1. Il Concetto di "Base Address"

Un puntatore a un oggetto contiene l'indirizzo di memoria del **primo byte** occupato dall'oggetto stesso. Non contiene "tutta la roba", ma solo il punto di partenza.

40.2 2. Layout Sequenziale (Esempio Pratico)

Immaginiamo una classe `Punto3D`:

```

1 class Punto3D {
2     int x; // 4 byte
3     int y; // 4 byte
4     int z; // 4 byte
5 };
6
7 Punto3D p; // Oggetto allocato all'indirizzo 0x100
8 Punto3D* ptr = &p; // Puntatore che vale 0x100

```

Mapa della Memoria:

Indirizzo	Offset	Contenuto
0x100	+0	x (inizio dell'oggetto) ← ptr punta qui
0x104	+4	y
0x108	+8	z
0x10C		(Fine dell'oggetto)

40.3 3. Come funziona l'accesso ai membri (->)

Quando scrivi `ptr->y`, il compilatore esegue un calcolo aritmetico nascosto basato sulla definizione della classe:

Indirizzo di y = Valore di ptr + Dimensione di x

$$0x104 = 0x100 + 4$$

40.4 4. Dimensione del Puntatore vs Oggetto

È fondamentale distinguere tra:

- `sizeof(p)`: La somma delle dimensioni dei membri interni (es. 12 byte). Rappresenta quanto spazio occupa la "casa".
- `sizeof(ptr)`: La dimensione dell'indirizzo di memoria (solitamente 8 byte su sistemi a 64-bit). Rappresenta quanto spazio occupa il "foglietto con l'indirizzo".

41 Contiguità e Data Alignment (Padding)

41.1 1. La Regola Generale

I membri di una classe vengono allocati in memoria in ordine sequenziale (contigui), basandosi sull'ordine di dichiarazione nel codice.

41.2 2. Il Padding (Spazio Vuoto)

Per motivi di efficienza hardware, la CPU accede alla memoria a blocchi (word) di 4 o 8 byte. I dati devono essere "allineati" ai loro indirizzi naturali. Se un dato non "combacia", il compilatore inserisce byte vuoti (**Padding**).

```
1 struct Brutta {  
2     char a;    // 1 byte  
3     // --- 3 byte di PADDING inseriti qui ---  
4     int b;     // 4 byte  
5     char c;    // 1 byte  
6     // --- 3 byte di PADDING inseriti qui ---  
7 };  
8 // sizeof(Brutta) = 12 byte (sprecati 6 byte!)
```

41.3 3. Ottimizzazione (Riordinamento)

Per risparmiare memoria, conviene dichiarare le variabili dalla più grande alla più piccola.

```
1 struct Bella {  
2     int b;     // 4 byte  
3     char a;    // 1 byte  
4     char c;    // 1 byte  
5     // --- 2 byte di PADDING finale per allineare la struttura ---  
6 };  
7 // sizeof(Bella) = 8 byte (sprecati solo 2)
```

41.4 4. Puntatori come Membri

Se un membro è un puntatore (`int* p`), solo l'indirizzo (8 byte) è memorizzato in modo contiguo all'interno dell'oggetto. I dati puntati risiedono altrove (spesso nello Heap) e non sono contigui all'oggetto.

41.5 Il Padding Finale (Tail Padding)

Anche riordinando le variabili, la dimensione totale della struttura deve essere sempre un multiplo dell'allineamento del suo membro più grande.

```
1 struct A {
2     int i; // 4 byte
3     char c; // 1 byte
4     // --> Qui vengono aggiunti 3 byte di PADDING
5 };
6 // sizeof(A) = 8 byte (non 5!)
```

Questo serve a garantire che in un array `A arr[2]`, il secondo elemento inizi correttamente allineato a un multiplo di 4.

42 Overloading e Tipi Primitivi

42.1 1. La Regola di Base

Non è possibile ridefinire gli operatori per i tipi fondamentali (built-in types) come `int`, `float`, `char`.

```
1 // ERRORE: Non puoi cambiare la somma tra due int nativi!
2 int operator+(int a, int b) { return (a > b) ? a : b; }
```

42.2 2. Wrapper Class

Bisogna creare una classe che contenga l'intero e ridefinire l'operatore per quella classe.

```
1 #include <iostream>
2 using namespace std;
3
4 class Intero {
5 private:
6     int valore;
7
8 public:
9     // Costruttore (permette conversioni implicite da int a Intero)
10    Intero(int v) : valore(v) {}
11
12    // Metodo per leggere il valore
13    int get() const { return valore; }
14
15    // --- OVERLOADING DEL + ---
16    // Definito come friend per simmetria
17    friend Intero operator+(const Intero& a, const Intero& b);
18
19    // Serve anche per permettere la stampa cout << oggetto
20    friend ostream& operator<<(ostream& os, const Intero& obj) {
21        os << obj.valore;
22        return os;
23    }
24 };
25
26 // Implementazione: il + ora restituisce il MAGGIORE (comportamento
27 // custom)
28 Intero operator+(const Intero& a, const Intero& b) {
```

```

28     return (a.valore > b.valore) ? a : b;
29 }
30
31 // MAIN DI ESEMPIO
32 /*
33 int main() {
34     Intero n1 = 10;
35     Intero n2 = 25;
36
37     // Qui usa il NOSTRO operatore +
38     Intero risultato = n1 + n2;
39
40     cout << risultato << endl; // Stampa 25!
41 }
42 */

```

43 Operatore di Assegnamento e Self-Assignment

43.1 1. Il Problema dell'Auto-Assegnazione (Aliasing)

Quando si scrive `obj = obj;`, l'oggetto sorgente e destinazione coincidono. Poiché l'assegnamento richiede la cancellazione della memoria precedente della destinazione, senza controlli si distruggerebbero i dati della sorgente prima di poterli copiare.

43.2 2. Il Test Standard

Il puntatore `this` rappresenta l'indirizzo dell'oggetto corrente (LHS), mentre `&c` è l'indirizzo del parametro (RHS).

```

1 Classe& operator=(const Classe& c) {
2     // TEST DI ALIASING
3     // "Se l'indirizzo mio e' uguale all'indirizzo di c..."
4     if (this == &c) {
5         return *this; // Non fare nulla, restituisci me stesso.
6     }
7
8     delete[] dati;          // 1. Pulisci
9     dati = new int[10];     // 2. Rialloca
10    // ... copia ...        // 3. Copia
11
12    return *this;
13 }

```

43.3 3. L'alternativa "Copy-and-Swap" (Strong Exception Safety)

Questo pattern evita il test esplicito delegando il lavoro al Costruttore di Copia. È considerato lo stile moderno C++ più robusto.

```

1 #include <algorithm> // per std::swap
2
3 class Classe {
4     int* dati;
5 public:

```

```

6 // 1. Costruttore di Copia (Standard)
7 Classe(const Classe& altra) { ... }
8
9 // 2. Funzione Swap (scambia i puntatori interni)
10 friend void swap(Classe& prima, Classe& seconda) {
11     using std::swap;
12     swap(prima.dati, seconda.dati); // Scambia solo gli indirizzi!
13     Veloce.
14 }
15
16 // 3. Operatore di Assegnamento (per valore!)
17 // Nota: il parametro 'c' e' passato PER VALORE, quindi e' GIA' una
18 // copia.
19 Classe& operator=(Classe c) {
20     // Scambio il mio contenuto (vecchio) con la copia (nuovo)
21     swap(*this, c);
22
23     // Alla fine della funzione, 'c' viene distrutto.
24     // E siccome 'c' ora contiene i miei vecchi dati (grazie allo
25     // swap),
26     // i miei vecchi dati vengono deallocati automaticamente dal
27     // distruttore di 'c'.
28
29     return *this;
30 }
31 };

```

44 Funzioni Membro Const

44.1 1. Significato

Aggiungendo `const` dopo la lista dei parametri, si dichiara che il metodo non modificherà alcuna variabile membro dell'oggetto (eccetto quelle marcate `mutable`).

```

1 class Esempio {
2     int x;
3 public:
4     // Promessa: non modifichero' x
5     int getX() const { return x; }
6
7     // Errore: non puoi modificare x in un metodo const!
8     // void setX(int val) const { x = val; }
9 };

```

44.2 2. La Matrice di Compatibilità

Chi può chiamare cosa?

Oggetto Chiamante	Metodo Const	Metodo Non-Const
Oggetto Non-Const	SÌ (Safe)	SÌ
Oggetto Const	SÌ	NO (Errore Compilazione)

44.3 3. Overloading su Const

È possibile definire due versioni dello stesso metodo per gestire diversamente l'accesso (Read-Write vs Read-Only). È il meccanismo usato ad esempio dall'operatore `[]` nei `vector` o nelle stringhe.

```
1 class Vettore {
2     int dati[100];
3 public:
4     // 1. Versione per oggetti MODIFICABILI
5     // Restituisce un reference modificabile (int&)
6     int& operator[](int index) {
7         return dati[index];
8     }
9
10    // 2. Versione per oggetti COSTANTI
11    // Restituisce un reference costante (const int&)
12    const int& operator[](int index) const {
13        return dati[index];
14    }
15 };
16
17 void test() {
18     Vettore v1;
19     v1[0] = 10; // Chiama la ver. 1 -> OK, modifica.
20
21     const Vettore v2;
22     // v2[0] = 10; -> ERRORE: Chiama la ver. 2 che restituisce un const
23     // int&.
24     int x = v2[0]; // OK: Chiama la ver. 2 per sola lettura.
```

44.4 4. Dietro le Quinte (Il Puntatore this)

La keyword `const` modifica il tipo del puntatore nascosto `this`:

- Metodo normale: `Classe* const this`
- Metodo `const`: `const Classe* const this` (Puntatore a oggetto costante)

45 Operatori di Conversione (User-Defined Conversion)

45.1 1. Definizione

Permettono di definire come un oggetto di una classe possa essere convertito (implicitamente o esplicitamente) in un altro tipo (primitivo o altra classe). È l'operazione inversa dei costruttori a un argomento.

45.2 2. Sintassi

- Non si specifica il tipo di ritorno (è dedotto dal nome dell'operatore).
- Solitamente sono funzioni `const`.
- Non accettano parametri.

```

1 class Frazione {
2     int n, d;
3 public:
4     Frazione(int n, int d) : n(n), d(d) {}
5
6     // Converte l'oggetto Frazione in un double
7     operator double() const {
8         return (double)n / d;
9     }
10 };

```

45.3 3. Il problema delle Conversioni Implicite

Se l'operatore non è marcato come `explicit`, il compilatore può usarlo in contesti inattesi, causando bug logici o ambiguità.

```

1 Frazione f(1, 2);
2 double x = f + 3.5; // OK: f diventa 0.5 implicitamente

```

45.4 4. La Keyword `explicit` (C++11)

Per evitare conversioni accidentali, si usa `explicit`. L'operatore verrà invocato solo con un cast statico o in contesti booleani diretti.

```

1 class SmartPtr {
2 public:
3     // Safe Bool Idiom moderno
4     explicit operator bool() const {
5         return ptr != nullptr;
6     }
7 private:
8     int* ptr;
9 };
10
11 SmartPtr p;
12 // bool b = p;           <- ERRORE: conversione implicita vietata!
13 if (p) { ... }          <- OK: contesto booleano esplicito.
14 bool b = (bool)p;       <- OK: cast esplicito.

```

46 Analisi Errore: Const Object vs Non-Const Method

46.1 1. Il Codice Incriminato

```

1 class Complesso {
2 public:
3     // Manca 'const' alla fine!
4     double reale() { return r; }
5 private:
6     double r, i;
7 };
8
9 const Complesso c1;
10 cout << c1.reale(); // <--- ERRORE DI COMPILAZIONE

```

46.2 2. L'Errore

Il compilatore genera un errore (es. *"discards qualifiers"*).

- **Motivo Logico:** L'oggetto `c1` è `const` (sola lettura). Il metodo `reale()` non è dichiarato `const`, quindi il compilatore assume che possa modificare lo stato dell'oggetto. La chiamata è vietata per sicurezza.
- **Motivo Tecnico:** Discrepanza nel puntatore `this`.
 - L'oggetto passa: `const Complesso* const this`
 - Il metodo accetta: `Complesso* const this`

Il C++ vieta la conversione implicita che rimuove il qualificatore `const`.

46.3 3. La Soluzione

Aggiungere `const` alla firma del metodo `getter`.

```
1 double reale() const { return r; } // ORA FUNZIONA
```

47 Membri Statici (Static Members)

47.1 1. Variabili Statiche (Data Members)

Una variabile dichiarata `static` all'interno di una classe:

- **Classe di Memorizzazione:** Static Storage Duration (Durata Statica).
- **Allocazione:** Risiede nel *Data Segment* globale, non nell'oggetto.
- **Molteplicità:** Esiste in **unica copia** condivisa da tutte le istanze della classe.
- **Sizeof:** Non influisce sulla dimensione dell'oggetto (`sizeof`).

Regola d'Oro dell'Inizializzazione: Dichiararla nella classe non basta (è solo una dichiarazione). Bisogna **definirla** e inizializzarla fuori dalla classe (solitamente nel file `.cpp`), altrimenti il linker darà errore.

```
1 class Contatore {
2 public:
3     static int conta; // Solo DICHIARAZIONE
4 };
5
6 // DEFINIZIONE (nel file .cpp o fuori dal main)
7 // Obbligatoria per allocare la memoria!
8 int Contatore::conta = 0;
```


47.2 2. Metodi Statici (Function Members)

Sono funzioni che appartengono alla classe ma non a un oggetto specifico.

- **Chiamata:** Si invocano con `NomeClasse::metodo()` (non serve un oggetto).
- **Limitazioni:**
 1. **Niente `this`:** Non ricevono il puntatore implicito `this`.
 2. **Accesso limitato:** Possono accedere SOLO a membri statici (variabili o funzioni). Non possono toccare membri non-statici (perché non saprebbero a quale oggetto riferirsi).

```
1 class Test {
2     int x;           // Variabile d'istanza
3     static int k;    // Variabile di classe (statica)
4 public:
5     static void funzioneStatica() {
6         k = 10;      // OK: k è condivisa
7         // x = 5;    // ERRORE: quale 'x'? Di quale oggetto? Manca 'this'
8     }
9 };
```

48 Significati della keyword 'static'

48.1 1. A livello di File (Variabili/Funzioni Globali)

Qui `static` influenza la **Visibilità (Linkage)**.

- **Internal Linkage:** La variabile/funzione è visibile **solo** all'interno del file sorgente (.cpp) in cui è definita.
- **Utilità:** Serve a incapsulare funzioni di supporto o costanti "private" del modulo, evitando conflitti di nome (Name Clashing) con altri file durante il linking.

48.2 2. A livello Locale (Dentro una Funzione)

Qui `static` influenza la **Durata (Lifetime)**.

- **Visibilità:** Resta locale al blocco (Scope). Nessuno da fuori può toccarla.
- **Memoria:** Viene allocata nel Data Segment (non nello Stack!) la prima volta che il flusso passa di lì.
- **Persistenza:** Mantiene il suo valore tra una chiamata e l'altra della funzione.

```
1 void contatore() {
2     static int c = 0; // Eseguita solo la prima volta!
3     c++;
4     cout << c << endl;
5 }
6 // Chiamata 1 -> stampa 1
7 // Chiamata 2 -> stampa 2 (si ricorda il valore!)
```

48.3 3. A livello di Classe

Qui **static** influenza l'**Appartenenza**. Indica che il membro appartiene alla classe e non alla singola istanza. La visibilità è controllata dai normali specificatori di accesso (public/private).

49 Return by Reference e Dangling References

49.1 1. L'Errore del Riferimento a Locale

Non restituire MAI un riferimento a una variabile locale o a un parametro passato per valore.

```
1 int& badMax(int a, int b) {  
2     // ERRORE GRAVE: 'a' muore quando la funzione termina.  
3     // Il chiamante riceve un riferimento a memoria deallocata.  
4     return (a > b) ? a : b;  
5 }
```

49.2 2. Funzioni come L-Value

Se una funzione restituisce un **Reference** (Type&), la chiamata alla funzione può apparire a sinistra di un assegnamento. La funzione rappresenta la variabile stessa, non il suo valore.

```
1 int& goodMax(int& a, int& b) {  
2     // OK: Restituisco l'alias dell'oggetto originale, che sopravvive.  
3     return (a > b) ? a : b;  
4 }  
5  
6 int main() {  
7     int x = 10, y = 20;  
8     goodMax(x, y) = 0; // Modifica y (il maggiore) portandolo a 0.  
9     // x = 10, y = 0  
10 }
```

49.3 3. Regola di Sicurezza

Una funzione che restituisce T& deve sempre ricevere parametri T& (o puntatori/variabili globali/membri di classe). Deve esserci una garanzia che l'oggetto riferito sopravviva alla fine della funzione.

50 Classi vs Struct in C++

50.1 1. Definizione

Una classe è un tipo di dato definito dall'utente che implementa il concetto di **Incapsulamento**, raggruppando dati (membri) e funzioni (metodi) che operano su di essi.

50.2 2. Differenze Tecniche

In C++, `class` e `struct` sono quasi intercambiabili. Le uniche differenze riguardano i valori di default:

Caratteristica	<code>struct</code>	<code>class</code>
Visibilità Membri Default	<code>public</code>	<code>private</code>
Ereditarietà Default	<code>public</code>	<code>private</code>

50.3 3. Esempio Comparativo

```
1 struct A {  
2     int x; // Public  
3 };  
4  
5 class B {  
6     int x; // Private  
7 public:    // Serve specificarlo per renderlo accessibile  
8     int y;  
9 };
```

50.4 4. Convenzioni d'Uso

- **Struct:** Usata per strutture dati passive ("Bag of data"), dove tutti i membri sono pubblici e non c'è logica di controllo (es. coordinate, pacchetti di rete).
- **Class:** Usata per entità attive che necessitano di proteggere il proprio stato interno (incapsulamento) e mantenere invarianti.

51 Polimorfismo in C++

51.1 1. Polimorfismo Statico (Early Binding)

La risoluzione della chiamata avviene al momento della compilazione.

- **Meccanismo:** Overloading di funzioni, Overloading di operatori, Template.
- **Vantaggi:** Efficienza massima (nessun overhead a runtime).
- **Svantaggi:** Meno flessibile, i tipi devono essere noti a priori.

```
1 // Esempio Overloading  
2 void disegna(int raggio) { ... } // Cerchio  
3 void disegna(int w, int h) { ... } // Rettangolo  
4  
5 // Il compilatore sa GIA' quale chiamare qui:  
6 disegna(10);
```

51.2 2. Polimorfismo Dinamico (Late Binding)

La risoluzione avviene durante l'esecuzione (Runtime). È il vero polimorfismo OOP.

- **Meccanismo:** Puntatori/Riferimenti alla classe base + Funzioni `virtual`.
- **Requisito:** La funzione nella classe base DEVE avere la parola chiave `virtual`.

```
1 class Animale {
2 public:
3     // "virtual" abilita il polimorfismo dinamico
4     virtual void verso() { cout << "..."; }
5 };
6
7 class Cane : public Animale {
8 public:
9     void verso() override { cout << "Bau!"; }
10 };
11
12 class Gatto : public Animale {
13 public:
14     void verso() override { cout << "Miao!"; }
15 };
16
17 void faiParlare(Animale* a) {
18     // QUI avviene la magia (Late Binding):
19     // Non guarda il tipo del puntatore (Animale*),
20     // ma il tipo dell'oggetto puntato (Cane o Gatto).
21     a->verso();
22 }
23
24 int main() {
25     Cane fido;
26     Gatto felix;
27
28     faiParlare(&fido); // Stampa "Bau!"
29     faiParlare(&felix); // Stampa "Miao!"
30 }
```

51.3 3. Tabella Comparativa

	Statico	Dinamico
Decisione	Compile-time	Run-time
Efficienza	Alta	Leggermente inferiore (Vtable)
Flessibilità	Media	Alta
Keywords	template, overloading	virtual, override

52 Quando ritornare un Reference (T&)

52.1 1. Per creare un L-Value (Accesso in Scrittura)

Usato quando la funzione rappresenta un accesso diretto a una variabile membro che si vuole poter modificare dall'esterno.

```

1 class Vettore {
2     int dati[100];
3 public:
4     // Ritorna int& -> Posso scriverci dentro!
5     int& at(int i) {
6         return dati[i];
7     }
8 };
9
10 Vettore v;
11 v.at(5) = 99; // OK: Scrive direttamente nella memoria di 'dati'

```

52.2 2. Per il "Method Chaining" (Cascata)

Tipico degli operatori di assegnamento e stream. Si ritorna `*this`.

```

1 class Finestra {
2 public:
3     // Ritorna un riferimento a Finestra (se stessa)
4     Finestra& setTitolo(string t) {
5         this->titolo = t;
6         return *this; // Restituisce l'oggetto corrente
7     }
8
9     Finestra& setDimensione(int w, int h) {
10        this->width = w; this->height = h;
11        return *this;
12    }
13 };
14
15 Finestra win;
16 // Concatenazione fluida:
17 win.setTitolo("Ciao").setDimensione(800, 600);

```

52.3 3. Per Ottimizzazione (Read-Only)

Per evitare la copia profonda (Deep Copy) di oggetti pesanti in lettura. Si usa `const T&`.

```

1 class Database {
2     vector<string> recordEnormi; // Immagina 1 GB di dati
3 public:
4     // EFFICIENTE: Non copia nulla, ritorna un alias.
5     const vector<string>& getDati() const {
6         return recordEnormi;
7     }
8
9     // LENTO: Creerebbe una copia completa del vettore!
10    // vector<string> getDatiLento() { return recordEnormi; }
11 };

```

52.4 4. IL DIVIETO ASSOLUTO

Mai ritornare un riferimento a una variabile **locale** della funzione.

```

1 int& errore() {
2     int x = 10;

```

```

3     return x; // CRASH: x viene distrutta al return!
4 }

```

53 Algoritmo di Overload Resolution

53.1 1. Individuazione dei Candidati

Il compilatore costruisce l'insieme delle *Candidate Functions*: tutte le funzioni visibili con lo stesso nome.

53.2 2. Funzioni Viabili (Viable Functions)

Vengono filtrate le funzioni che:

- Hanno lo stesso numero di parametri (o parametri con valori di default).
- Hanno tipi di parametri convertibili in quelli richiesti.

53.3 3. Selezione della Migliore (Best Match)

Viene scelta la funzione che richiede la "conversione migliore" per ogni argomento. La gerarchia è:

1. **Exact Match:** Nessuna conversione o conversioni banali (Array-to-pointer, T to const T).
2. **Promotion:**
 - Integer Promotion: char/short/bool → int
 - Float Promotion: float → double
3. **Standard Conversion:** (es. int → double, Base* → Derived*, void*).
4. **User-Defined Conversion:** (Costruttori impliciti, operatori di cast).
5. **Ellipsis (...):** Argomenti variabili C-style.

53.4 4. Ambiguità

Se due funzioni sono allo stesso livello di priorità e nessuna è "strettamente migliore" dell'altra, il compilatore genera un errore: *"Call to 'f' is ambiguous"*.

54 Strutture Dati e Algoritmi

54.1 3. Lista Doppia e Stampa Ricorsiva Inversa

54.1.1 1. La Struttura (Nodo Doppio)

Rispetto alla lista singola, aggiungiamo il puntatore `prev`. Questo permette di scorrere la lista in entrambe le direzioni.

```

1 struct Nodo {
2     int info;
3     Nodo* next;
4     Nodo* prev; // Puntatore al nodo precedente
5 };

```

54.1.2 2. La Funzione Ricorsiva (Il "Trucco")

L'algoritmo sfrutta il funzionamento dello Stack:

1. La funzione chiama se stessa scorrendo in avanti (**next**) fino alla fine.
2. Quando tocca il **nullptr** (caso base), torna indietro.
3. L'istruzione di stampa è messa **DOPO** la chiamata ricorsiva. Quindi viene eseguita solo mentre la funzione "ritorna" (unwinding), stampando dall'ultimo al primo.

```

1 // Funzione richiesta
2 void stampaInversaRicorsiva(Nodo* p) {
3     // 1. Caso Base: Se sono fuori dalla lista, mi fermo.
4     if (p == nullptr) {
5         return;
6     }
7
8     // 2. Passo Ricorsivo: Vado avanti fino in fondo PRIMA di fare altro
9     stampaInversaRicorsiva(p->next);
10
11    // 3. Azione al Ritorno: Stampo il valore
12    // Questa riga viene eseguita solo quando la chiamata sopra ritorna
13    cout << p->info << " ";
14 }

```

54.1.3 3. Implementazione Completa e Test

Ecco un programma completo che include anche l'inserimento per poter testare il codice.

```

1 #include <iostream>
2 using namespace std;
3
4 struct Nodo {
5     int info;
6     Nodo* next;
7     Nodo* prev;
8 };
9
10 // Funzione helper per creare la lista (Inserimento in Testa)
11 void inserisci(Nodo*& testa, int valore) {
12     Nodo* nuovo = new Nodo;
13     nuovo->info = valore;
14     nuovo->next = testa;
15     nuovo->prev = nullptr;
16
17     if (testa != nullptr) {
18         testa->prev = nuovo; // Collego il vecchio primo al nuovo
19     }
20     testa = nuovo; // Aggiorno la testa

```

```

21 }
22
23 void stampaInversaRicorsiva(Nodo* p) {
24     if (p == nullptr) return;
25     stampaInversaRicorsiva(p->next);
26     cout << p->info << " ";
27 }
28
29 int main() {
30     Nodo* lista = nullptr;
31
32     // Inserisco: 10, poi 20, poi 30
33     // La lista sara': 30 <-> 20 <-> 10
34     inserisci(lista, 10);
35     inserisci(lista, 20);
36     inserisci(lista, 30);
37
38     cout << "Stampa Inversa (Ricorsiva): ";
39     stampaInversaRicorsiva(lista);
40     // Output atteso: 10 20 30
41     // (Nota: la lista fisica e' 30-20-10, quindi l'inverso e' 10-20-30)
42
43     return 0;
44 }

```

54.1.4 Analisi della Complessità

- **Tempo:** $O(n)$. Dobbiamo visitare ogni nodo una volta.
- **Spazio:** $O(n)$. Nonostante non usiamo strutture dati extra, la ricorsione occupa memoria nello **Stack di sistema**. Se la lista ha 1 milione di elementi, il programma andrà in *Stack Overflow*.

54.2 Min e Max in Lista Ordinata

Dato che la lista è ordinata in modo crescente (o non decrescente):

- $min = head \rightarrow info$ (Complessità $O(1)$).
- $max = tail \rightarrow info$ (Complessità $O(n)$ perché dobbiamo scorrere fino in fondo).

54.2.1 Gestione Lista Vuota

È il caso critico (Edge Case). Non possiamo accedere a `head->info` se `head` è `nullptr` (Crash immediato). Soluzione: lanciamo un'eccezione o ritorniamo un codice di errore/-valore sentinella. Qui usiamo `std::runtime_error` per pulizia.

54.2.2 Implementazione

```

1 #include <iostream>
2 #include <stdexcept> // Per gestire gli errori
3 using namespace std;
4
5 struct Nodo {

```



```

6     int info;
7     Nodo* next;
8 };
9
10 // 1. Trova Minimo (Il Primo elemento)
11 // Complessita': O(1) - Istantaneo
12 int trovaMin(Nodo* testa) {
13     if (testa == nullptr) {
14         throw runtime_error("Errore: Lista vuota, nessun minimo.");
15     }
16     // Essendo ordinata, il piu' piccolo e' subito qui.
17     return testa->info;
18 }
19
20 // 2. Trova Massimo (L'Ultimo elemento)
21 // Complessita': O(n) - Devo arrivare in fondo
22 int trovaMax(Nodo* testa) {
23     if (testa == nullptr) {
24         throw runtime_error("Errore: Lista vuota, nessun massimo.");
25     }
26
27     Nodo* curr = testa;
28     // Scorro finche' esiste un nodo successivo
29     while (curr->next != nullptr) {
30         curr = curr->next;
31     }
32
33     // Ora curr punta all'ultimo nodo
34     return curr->info;
35 }

```

54.2.3 Ottimizzazione Possibile

Se avessimo usato una struttura con il puntatore alla coda (tail pointer) mantenuto aggiornato durante gli inserimenti:

```

1 struct Lista {
2     Nodo* head;
3     Nodo* tail; // Puntatore diretto all'ultimo
4 };

```

Anche la funzione `trovaMax` diventerebbe $O(1)$ (costante), accedendo direttamente a `tail->info`.

55 Selection Sort

55.1 1. Principio di Funzionamento

L'algoritmo divide l'array in due parti: la parte sinistra (ordinata) e la parte destra (disordinata).

1. Cerca il **minimo assoluto** nella parte disordinata.
2. Lo **scambia** con il primo elemento della parte disordinata.
3. Avanza il confine tra le due parti.

55.2 2. Implementazione C++

```
1 #include <algorithm> // Necessario per std::swap
2
3 void selectionSort(int arr[], int n) {
4     // 1. Sposta il confine della parte ordinata
5     for (int i = 0; i < n - 1; i++) {
6
7         // Trova il minimo nella parte non ordinata (da i a n-1)
8         int min_idx = i;
9         for (int j = i + 1; j < n; j++) {
10             if (arr[j] < arr[min_idx]) {
11                 min_idx = j; // Aggiorno l'indice del nuovo minimo
12             }
13         }
14
15         // 2. Scambia il minimo trovato con l'elemento corrente
16         if (min_idx != i) {
17             std::swap(arr[i], arr[min_idx]);
18         }
19     }
20 }
```

55.3 3. Analisi della Complessità

- **Tempo (Best/Avg/Worst):** $O(n^2)$. *Motivo:* I due cicli nidificati vengono eseguiti sempre, anche se l'array è già ordinato. Fa sempre $\approx \frac{n^2}{2}$ confronti.
- **Spazio:** $O(1)$ (In-place).
- **Scambi (Swaps):** $O(n)$. *Punto di forza:* Esegue al massimo $N - 1$ scritture in memoria. È ottimo quando scrivere in memoria è un'operazione costosa (es. memoria Flash).
- **Stabilità:** **NO**. Lo scambio a lunga distanza può alterare l'ordine relativo di elementi uguali.

56 Bubble Sort

56.1 1. Principio di Funzionamento

L'algoritmo esegue scansioni ripetute dell'array. In ogni scansione, confronta le coppie adiacenti $(j, j + 1)$.

- Se $v[j] > v[j + 1]$: Esegue lo **Swap**.
- Al termine del primo ciclo, il massimo assoluto si trova nell'ultima posizione $(N - 1)$.
- Al termine del secondo ciclo, il secondo massimo è in posizione $N - 2$.
- Si ripete riducendo il range di 1 ogni volta.

56.2 2. Implementazione C++ (Ottimizzata)

La versione "ingenua" fa sempre N^2 controlli. La versione ottimizzata usa una **flag (sentinella)**: se in un passaggio non avviene nessuno scambio, significa che l'array è già ordinato e ci fermiamo subito.

```
1 void bubbleSort(int arr[], int n) {
2     bool swapped; // Flag per ottimizzazione
3
4     // Ciclo esterno: conta le passate
5     for (int i = 0; i < n - 1; i++) {
6         swapped = false;
7
8         // Ciclo interno: confronta le coppie
9         // Nota: j arriva fino a n-i-1 perche' gli ultimi i elementi
10        // sono gia' "bolliti" (ordinati in fondo).
11        for (int j = 0; j < n - i - 1; j++) {
12            if (arr[j] > arr[j+1]) {
13                // Sono disordinati: Scambio!
14                std::swap(arr[j], arr[j+1]);
15                swapped = true; // Ho fatto almeno uno scambio
16            }
17        }
18
19        // Se non ho fatto scambi in questo giro, ho finito!
20        if (swapped == false)
21            break;
22    }
23 }
```

56.3 3. Analisi della Complessità

- **Caso Peggiore:** $O(n^2)$. (Array ordinato al contrario).
- **Caso Medio:** $O(n^2)$.
- **Caso Migliore:** $O(n)$. (Grazie alla flag **swapped**, se l'array è già ordinato fa solo un giro di controllo e esce).
- **Spazio:** $O(1)$ (In-place).
- **Stabilità: SÌ.** *Motivo:* Se due elementi sono uguali ($5_a, 5_b$), la condizione $arr[j] > arr[j + 1]$ è falsa, quindi non vengono scambiati. 5_a rimarrà sempre prima di 5_b .

56.4 Ricerca Lineare vs Ricerca Binaria

56.4.1 1. Ricerca Lineare (Sequenziale)

approccio forza bruta. Si scorre l'array dall'inizio alla fine finché non si trova l'elemento o si termina la lista.

- **Precondizioni:** Nessuna. Funziona su array **disordinati** o ordinati.
- **Struttura Dati:** Array o Liste Concatenate.

- **Complessità:** $O(n)$. Nel caso peggiore (elemento non presente o in fondo), bisogna controllare tutti gli N elementi.

```

1 int linearSearch(int arr[], int n, int target) {
2     for (int i = 0; i < n; i++) {
3         if (arr[i] == target) {
4             return i; // Trovato all'indice i
5         }
6     }
7     return -1; // Non trovato
8 }

```

56.4.2 2. Ricerca Binaria (Dicotomica)

È l'approccio "divide et impera". Ad ogni passo, si confronta l'elemento centrale con il target e si scarta metà dell'array.

- **Precondizioni:** L'array **DEVE** essere ordinato. Se non lo è, l'algoritmo fallisce.
- **Struttura Dati:** Solo Array (o Vector) con accesso casuale ($O(1)$). Non efficiente su Liste Linkate.
- **Complessità:** $O(\log_2 n)$. Estremamente veloce su grandi dataset.

```

1 int binarySearch(int arr[], int n, int target) {
2     int low = 0;
3     int high = n - 1;
4
5     while (low <= high) {
6         // Calcolo sicuro del centro (evita overflow)
7         int mid = low + (high - low) / 2;
8
9         if (arr[mid] == target)
10            return mid; // Trovato
11
12        if (arr[mid] < target)
13            low = mid + 1; // Cerca nella metà destra
14        else
15            high = mid - 1; // Cerca nella metà sinistra
16    }
17    return -1; // Non trovato
18 }

```

56.4.3 3. Confronto Prestazionale

Il divario di efficienza cresce esponenzialmente con la dimensione dei dati.

Elementi (N)	Ricerca Lineare (N)	Ricerca Binaria ($\log_2 N$)
10	10 controlli	4 controlli
1.000	1.000 controlli	10 controlli
1.000.000	1.000.000 controlli	20 controlli
1 Miliardo	1 Miliardo di controlli	30 controlli

56.5 Capacità Coda Circolare (Il Caso $v[10]$)

Data la struttura:

```
1 struct Coda {  
2     int v[10];  
3     int testa;  
4     int coda;  
5 };
```

56.5.1 Domanda 1: Quanti elementi si possono memorizzare?

Risposta: Al massimo **9 elementi** ($N - 1$).

56.5.2 Domanda 2: Perché?

A causa del **problema dell'ambiguità tra coda vuota e coda piena**.

- **Condizione di Coda Vuota:** `testa == coda`.
- Se riempiamo tutti i 10 elementi, l'indice `coda` farebbe il "giro completo" tornando a coincidere con `testa`.
- Il sistema si troverebbe di nuovo con `testa == coda` e non saprebbe distinguere se la struttura contiene 0 elementi o 10.

Per risolvere il problema si lascia sempre una cella libera cuscinetto. La condizione di pieno diventa quindi:

$$(coda + 1) \% 10 == testa$$

56.5.3 Domanda 3: Cosa fare per memorizzarne 10?

Bisogna modificare la struttura aggiungendo una variabile che conti esplicitamente gli elementi presenti.

Nuova Struttura:

```
1 struct Coda {  
2     int v[10];  
3     int testa;  
4     int coda;  
5     int num_elementi; // <--- Soluzione  
6 };
```

Con questa variabile aggiuntiva:

- Coda Vuota: `num_elementi == 0`
- Coda Piena: `num_elementi == 10`

Non basandoci più sul confronto tra `testa` e `coda` per capire se è piena, possiamo usare tutti gli slot dell'array.

57 Definizione di Algoritmo

Un algoritmo è un procedimento sistematico per risolvere un problema. Matematicamente, è una funzione che trasforma un Input in un Output:

$$f : Input \rightarrow Output$$

57.1 5 Proprietà (F.A.R.G.D.)

1. **Finitezza**: Deve terminare in un tempo finito.
2. **Assenza di ambiguità (Determinismo)**: Ogni istruzione deve essere precisa e interpretabile in un solo modo.
3. **Realizzabilità**: Istruzioni eseguibili materialmente dall'esecutore.
4. **Generalità**: Deve risolvere una classe di problemi, non una singola istanza.
5. **Dati (Input/Output)**: Deve avere un'interfaccia definita.

58 Gestione File (<fstream>)

Per operare sui file si utilizzano le classi `ofstream` (scrittura) e `ifstream` (lettura).

58.1 Scrittura Formattata

Si usa l'operatore « esattamente come con `cout`. I manipolatori di `<iomanip>` permettono di allineare i dati nel file.

```
1 ofstream out("dati.txt");
2 if(out.is_open()) {
3     // Scrive: "Nome           Punti"
4     out << left << setw(15) << "Nome" << "Punti" << endl;
5     // Scrive: "Mario          100"
6     out << left << setw(15) << "Mario" << 100 << endl;
7     out.close();
8 }
```

58.2 Lettura Riga per Riga (Pattern Standard)

Il metodo più sicuro per leggere un file di testo è usare `getline` all'interno della condizione di un ciclo `while`.

```
1 ifstream in("dati.txt");
2 string riga;
3
4 if (in) { // Check se aperto correttamente
5     // Il ciclo continua finché non si raggiunge EOF (End Of File)
6     while (getline(in, riga)) {
7         cout << riga << endl; // Elabora la riga
8     }
9     in.close();
10 }
```

58.3 Modalità di Apertura (File Modes)

Quando si apre un file, si può specificare una modalità come secondo parametro:

- `ios::in`: Apre in lettura (default per `ifstream`).
- `ios::out`: Apre in scrittura (default per `ofstream`). Sovrascrive il file.
- `ios::app`: (**A**ppend) Scrive in coda al file senza cancellare il contenuto precedente.
- `ios::binary`: Apre in modalità binaria (non traduce i caratteri speciali come `\n`).

Esempio Append:

```
1 // Apre per aggiungere dati alla fine, senza cancellare
2 ofstream f("log.txt", ios::app);
```

59 Ereditarietà (Inheritance)

L'ereditarietà permette a una classe *Derivata* di acquisire membri da una classe *Base*.

59.1 1. Specificatori di Accesso all'Ereditarietà

Sintassi: `class Derivata : public Base { ... };`

Nella Base	Eredità Public	Eredità Protected	Eredità Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Inaccessibile	Inaccessibile	Inaccessibile

59.2 2. Costruttori e Distruttori

L'ordine di chiamata è fisso e speculare:

- **Costruzione:** $\text{Base} \rightarrow \text{Derivata}$.
- **Distruzione:** $\text{Derivata} \rightarrow \text{Base}$.

Passaggio parametri al costruttore Base:

```
1 class Base {
2 public:
3     Base(int x) { ... }
4 };
5
6 class Derivata : public Base {
7 public:
8     // Derivata deve chiamare esplicitamente il costruttore Base
9     Derivata(int x, int y) : Base(x) {
10         // uso y per inizializzare Derivata
11     }
12 };
```

59.3 3. Polimorfismo e Override

Per permettere alla classe derivata di ridefinire un comportamento e sfruttare il puntatore alla classe base, si usano le funzioni `virtual`.

```
1 class Animale {
2 public:
3     virtual void verso() { cout << "..."; }
4     // FONDAMENTALE: Distruttore virtuale per evitare memory leak
5     virtual ~Animale() {}
6 };
7
8 class Cane : public Animale {
9 public:
10    // 'override' assicura che stiamo sovrascrivendo correttamente
11    void verso() override { cout << "Bau"; }
12 };
13
14 // Utilizzo Polimorfico
15 Animale* a = new Cane();
16 a->verso(); // Stampa "Bau" grazie a 'virtual'
17 delete a;   // Chiama ~Cane() poi ~Animale() grazie a virtual dtor
```

59.4 4. Classi Astratte e Metodi Puri

Una classe è astratta se ha almeno un metodo puramente virtuale (= 0). Non può essere istanziata. Serve come interfaccia.

```
1 class Forma {
2 public:
3     // Metodo puramente virtuale
4     virtual double area() = 0;
5 };
6
7 class Cerchio : public Forma {
8 public:
9     double r;
10    // Obbligatorio implementarlo per rendere Cerchio concreto
11    double area() override { return 3.14 * r * r; }
12 };
```

60 Template e Standard Template Library (STL)

60.1 1. I Template (Programmazione Generica)

I template permettono di definire funzioni o classi indipendenti dal tipo di dato specifico. Il tipo viene definito come parametro (spesso indicato con `T`) e risolto a tempo di compilazione.

Sintassi Base:

```
1 // Template di Funzione
2 template <typename T>
3 void swap_values(T& a, T& b) {
4     T temp = a;
5     a = b;
6     b = temp;
7 }
```



```

7 }
8
9 // Template di Classe
10 template <typename T>
11 class Contenitore {
12     T elemento;
13 public:
14     Contenitore(T el) : elemento(el) {}
15 };

```

60.2 2. Contenitori STL Principali

La STL fornisce strutture dati generiche ottimizzate.

60.2.1 A. Vector vs List (Confronto Critico)

Caratteristica	<code>std::vector</code>	<code>std::list</code>
Struttura	Array Dinamico (Contiguo)	Lista Doppia Linkata
Accesso	Casuale $O(1)$ (es. <code>v[i]</code>)	Sequenziale $O(n)$
Inserimento (Coda)	Veloce $O(1)$	Veloce $O(1)$
Inserimento (Centro)	Lento $O(n)$ (shift dati)	Veloce $O(1)$ (solo puntatori)
Utilizzo	Default choice (Cache friendly)	Solo se insert/delete frequenti in mezzo

60.2.2 B. Container Adapters (Stack e Queue)

Questi contenitori limitano l'accesso ai dati per imporre una logica specifica. Non usano iteratori standard.

- **`std::stack` (LIFO - Last In First Out):** Simula una pila.
 - `push(e)`: Inserisce in cima.
 - `pop()`: Rimuove dalla cima.
 - `top()`: Legge l'elemento in cima.
- **`std::queue` (FIFO - First In First Out):** Simula una coda d'attesa.
 - `push(e)`: Inserisce in fondo (back).
 - `pop()`: Rimuove dalla testa (front).
 - `front()`: Legge l'elemento in testa.

60.2.3 Esempio di utilizzo Vector

```

1 #include <vector>
2 std::vector<int> v;
3 v.push_back(10); // Aggiunge in coda
4 v.push_back(20);
5 // Accesso come array
6 std::cout << v[0]; // Stampa 10
7 // Dimensione
8 std::cout << v.size(); // Stampa 2

```

61 Sistemi di Tipizzazione

Un sistema di tipi è un insieme di regole che assegna una proprietà (il tipo) ai costrutti del programma (variabili, espressioni, funzioni).

61.1 1. Classificazione

- **Tipizzazione Statica (Static Typing):** Il controllo dei tipi avviene a *Tempo di Compilazione* (Compile-time).
 - Le variabili devono essere dichiarate con un tipo.
 - Gli errori di tipo impediscono la creazione dell'eseguibile.
 - Esempi: C, C++, Java.
- **Tipizzazione Dinamica (Dynamic Typing):** Il controllo avviene a *Tempo di Esecuzione* (Runtime).
 - Le variabili assumono il tipo del valore che contengono in quel momento.
 - Esempi: Python, JavaScript.

61.2 2. Il Caso C++

Il C++ è un linguaggio **Staticamente Tipizzato**. Ogni variabile ha un tipo determinato alla compilazione e non può cambiarlo.

61.2.1 Type Inference (C++11 'auto')

La keyword `auto` permette al compilatore di dedurre il tipo dall'inizializzazione. **Nota:** Non è tipizzazione dinamica! Il tipo viene fissato per sempre alla riga di dichiarazione.

```
1 auto x = 5;           // x è int
2 auto y = 3.14;        // y è double
3 // x = "test";        // ERRORE DI COMPILAZIONE: x è un int!
```

61.2.2 RTTI (Run-Time Type Information)

Sebbene statico, il C++ offre meccanismi per ispezionare i tipi a runtime, utili nel polimorfismo:

- `typeid(obj)`: Restituisce informazioni sul tipo dell'oggetto.
- `dynamic_cast<T*>(ptr)`: Tenta di convertire un puntatore base in derivato a runtime. Se fallisce, restituisce `nullptr`.

62 Incapsulamento e Funzioni Virtuali

62.1 1. Incapsulamento (Information Hiding)

È il meccanismo che raggruppa dati e metodi all'interno di una classe, nascondendo lo stato interno all'esterno. Si realizza tramite i livelli di accesso:

- **private:** Accessibile solo dai metodi della classe stessa. (Default per le `class`).
- **protected:** Accessibile dalla classe stessa e dalle classi derivate.
- **public:** Accessibile da chiunque. (Default per le `struct`).

Obiettivo: Garantire l'integrità dei dati impedendo modifiche arbitrarie e disaccoppiare l'implementazione dall'interfaccia.

62.2 2. Funzioni Virtuali (Meccanismo)

Una funzione `virtual` attiva il **Dynamic Binding** (collegamento dinamico). La risoluzione della chiamata a funzione viene posticipata al tempo di esecuzione (Runtime) basandosi sul *tipo dinamico* dell'oggetto puntato, non sul *tipo statico* del puntatore.

62.2.1 Come funziona: La V-Table

Per ogni classe che contiene metodi virtuali, il compilatore genera una **Virtual Table (vtable)**: un array di puntatori alle funzioni implementate da quella classe.

- Ogni oggetto istanziato contiene un puntatore nascosto (`vptr`) che punta alla vtable della sua classe.
- Una chiamata `obj->metodo()` viene tradotta in: `*(obj->vptr[index])()`.
- Questo aggiunge un leggero overhead (costo prestazionale) rispetto alle funzioni non virtuali.

62.2.2 Keywords Correlate (C++11)

- **override:** Esplicita l'intenzione di sovrascrivere un metodo virtuale della base. Genera errore se la firma non corrisponde.
- **final:** Impedisce che un metodo virtuale venga ulteriormente sovrascritto nelle classi derivate successive.

```

1 class Base {
2 public:
3     virtual void print() { cout << "Base"; }
4 };
5
6 class Derivata : public Base {
7 public:
8     void print() override { cout << "Derivata"; }
9 };
10
11 // Esempio V-Table in azione
12 Base* b = new Derivata();
13 b->print(); // Stampa "Derivata" (grazie a virtual)
14 // Senza virtual, avrebbe stampato "Base"

```